

# Diseño Material 3 + Workflows, Agents & Logs

Exportación completa del rediseño de Helix Orchestrator: sistema de idiomas ES/EN, navegación compartida, dashboard rediseñado, y las tres páginas nuevas (Workflows, Agents, Logs) con sus componentes y rutas.

- Layout & estilos globales
- Sistema de idiomas (ES/EN)
- Nav, footer e íconos compartidos
- Dashboard rediseñado
- Datos y permisos
- Página Workflows
- Página Agents
- Página Logs

# Layout & estilos globales

## src/app/layout.tsx

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import { TooltipProvider } from "@components/ui/tooltip";
import { LanguageProvider } from "@lib/i18n/language-provider";
import "./globals.css";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "Helix Orchestrator · Multi-Agent Content System",
  description:
    "Orquestador multi-agente para análisis de contenido: extracción con confidence score, SEO, verificación de hechos, recomendaciones y revisión cruzada.",
};

export default function RootLayout({ children }: LayoutProps<"/>) {
  return (
    <html
      lang="es"
      className={`dark ${geistSans.variable} ${geistMono.variable} h-full antialiased`}
    >
      <head>
        <link rel="preconnect" href="https://fonts.googleapis.com" />
        <link
          href="https://fonts.googleapis.com/css2?family=Material+Symbols+Outlined:opsz,wght,FILL,GRAD@20..48,100..700,0..1,-50..200&display=swap"
          rel="stylesheet"
        />
      </head>
      <body className="min-h-full flex flex-col selection:bg-primary selection:text-on-primary">
        <LanguageProvider>
          <TooltipProvider>{children}</TooltipProvider>
        </LanguageProvider>
      </body>
    </html>
  );
}
```

## src/app/globals.css

```
@import "tailwindcss";
@import "tw-animate-css";
@import "shadcn/tailwind.css";

@custom-variant dark (&:is(.dark *));
```

```

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans), ui-sans-serif, system-ui, sans-serif;
  --font-mono: var(--font-geist-mono), ui-monospace, monospace;
  --font-heading: var(--font-geist-sans), ui-sans-serif, system-ui, sans-serif;
  --color-surface: var(--surface);
  --color-surface-dim: var(--surface-dim);
  --color-surface-bright: var(--surface-bright);
  --color-surface-container-lowest: var(--surface-container-lowest);
  --color-surface-container-low: var(--surface-container-low);
  --color-surface-container: var(--surface-container);
  --color-surface-container-high: var(--surface-container-high);
  --color-surface-container-highest: var(--surface-container-highest);
  --color-on-surface: var(--on-surface);
  --color-on-surface-variant: var(--on-surface-variant);
  --color-on-background: var(--on-background);
  --color-outline: var(--outline);
  --color-outline-variant: var(--outline-variant);
  --color-tertiary: var(--tertiary);
  --color-tertiary-container: var(--tertiary-container);
  --color-on-tertiary: var(--on-tertiary);
  --color-on-tertiary-container: var(--on-tertiary-container);
  --color-error-container: var(--error-container);
  --color-on-error: var(--on-error);
  --color-on-error-container: var(--on-error-container);
  --color-on-primary-container: var(--on-primary-container);
  --color-primary-container: var(--primary-container);
  --color-sidebar-ring: var(--sidebar-ring);
  --color-sidebar-border: var(--sidebar-border);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar: var(--sidebar);
  --color-chart-5: var(--chart-5);
  --color-chart-4: var(--chart-4);
  --color-chart-3: var(--chart-3);
  --color-chart-2: var(--chart-2);
  --color-chart-1: var(--chart-1);
  --color-ring: var(--ring);
  --color-input: var(--input);
  --color-border: var(--border);
  --color-destructive: var(--destructive);
  --color-accent-foreground: var(--accent-foreground);
  --color-accent: var(--accent);
  --color-muted-foreground: var(--muted-foreground);
  --color-muted: var(--muted);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-secondary: var(--secondary);
  --color-primary-foreground: var(--primary-foreground);
  --color-primary: var(--primary);
  --color-popover-foreground: var(--popover-foreground);
  --color-popover: var(--popover);
  --color-card-foreground: var(--card-foreground);
  --color-card: var(--card);
  --radius-sm: calc(var(--radius) * 0.6);
  --radius-md: calc(var(--radius) * 0.8);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) * 1.4);
  --radius-2xl: calc(var(--radius) * 1.8);

```

```

--radius-3xl: calc(var(--radius) * 2.2);
--radius-4xl: calc(var(--radius) * 2.6);
}

:root {
  --background: oklch(1 0 0);
  --foreground: oklch(0.145 0 0);
  --card: oklch(1 0 0);
  --card-foreground: oklch(0.145 0 0);
  --popover: oklch(1 0 0);
  --popover-foreground: oklch(0.145 0 0);
  --primary: oklch(0.205 0 0);
  --primary-foreground: oklch(0.985 0 0);
  --secondary: oklch(0.97 0 0);
  --secondary-foreground: oklch(0.205 0 0);
  --muted: oklch(0.97 0 0);
  --muted-foreground: oklch(0.556 0 0);
  --accent: oklch(0.97 0 0);
  --accent-foreground: oklch(0.205 0 0);
  --destructive: oklch(0.577 0.245 27.325);
  --border: oklch(0.922 0 0);
  --input: oklch(0.922 0 0);
  --ring: oklch(0.708 0 0);
  --chart-1: oklch(0.87 0 0);
  --chart-2: oklch(0.556 0 0);
  --chart-3: oklch(0.439 0 0);
  --chart-4: oklch(0.371 0 0);
  --chart-5: oklch(0.269 0 0);
  --radius: 0.625rem;
  --sidebar: oklch(0.985 0 0);
  --sidebar-foreground: oklch(0.145 0 0);
  --sidebar-primary: oklch(0.205 0 0);
  --sidebar-primary-foreground: oklch(0.985 0 0);
  --sidebar-accent: oklch(0.97 0 0);
  --sidebar-accent-foreground: oklch(0.205 0 0);
  --sidebar-border: oklch(0.922 0 0);
  --sidebar-ring: oklch(0.708 0 0);
}

.dark {
  --background: #0e1320;
  --foreground: #dee2f5;
  --card: #1a1f2d;
  --card-foreground: #dee2f5;
  --popover: #1a1f2d;
  --popover-foreground: #dee2f5;
  --primary: #4cd7f6;
  --primary-foreground: #003640;
  --secondary: #3131c0;
  --secondary-foreground: #e1e0ff;
  --muted: #252a38;
  --muted-foreground: #bcc9cd;
  --accent: #303443;
  --accent-foreground: #dee2f5;
  --destructive: #ffb4ab;
  --border: #3d494c;
  --input: #3d494c;
  --ring: #4cd7f6;
  --chart-1: #4cd7f6;
  --chart-2: #4edea3;
  --chart-3: #c0c1ff;
  --chart-4: #ffb4ab;

```

```

--chart-5: #6ffbbe;
--sidebar: #161b29;
--sidebar-foreground: #dee2f5;
--sidebar-primary: #4cd7f6;
--sidebar-primary-foreground: #003640;
--sidebar-accent: #252a38;
--sidebar-accent-foreground: #dee2f5;
--sidebar-border: #3d494c;
--sidebar-ring: #4cd7f6;

--surface: #0e1320;
--surface-dim: #0e1320;
--surface-bright: #343948;
--surface-container-lowest: #090e1b;
--surface-container-low: #161b29;
--surface-container: #1a1f2d;
--surface-container-high: #252a38;
--surface-container-highest: #303443;
--on-surface: #dee2f5;
--on-surface-variant: #bcc9cd;
--on-background: #dee2f5;
--outline: #869397;
--outline-variant: #3d494c;
--tertiary: #4edea3;
--tertiary-container: #1bbd85;
--on-tertiary: #003824;
--on-tertiary-container: #00452e;
--error-container: #93000a;
--on-error: #690005;
--on-error-container: #ffdad6;
--primary-container: #06b6d4;
--on-primary-container: #00424f;
}

@layer base {
  * {
    @apply border-border outline-ring/50;
  }
  html {
    @apply font-sans;
  }
  body {
    @apply bg-background text-foreground antialiased;
    background-image:
      radial-gradient(1200px 500px at 10% -10%, oklch(0.45 0.12 220 / 0.28), transparent 55%),
      radial-gradient(900px 420px at 100% 0%, oklch(0.4 0.1 280 / 0.18), transparent 50%),
      linear-gradient(180deg, oklch(0.2 0.035 255), oklch(0.17 0.03 255));
    min-height: 100%;
  }
}

.helix-grid {
  background-image: linear-gradient(oklch(0.92 0.02 250 / 4%) 1px, transparent 1px),
    linear-gradient(90deg, oklch(0.92 0.02 250 / 4%) 1px, transparent 1px);
  background-size: 42px 42px;
}

.helix-scan {
  animation: helix-pulse 1.6s ease-in-out infinite;
}

@keyframes helix-pulse {

```

```

0%,
100% {
  opacity: 0.45;
}
50% {
  opacity: 1;
}
}

.glow-primary {
  box-shadow: 0 0 12px rgba(76, 215, 246, 0.25);
}

.glow-text-primary {
  text-shadow: 0 0 8px rgba(76, 215, 246, 0.4);
}

.material-symbols-outlined {
  font-family: "Material Symbols Outlined";
  font-weight: normal;
  font-style: normal;
  font-size: 20px;
  line-height: 1;
  letter-spacing: normal;
  text-transform: none;
  display: inline-block;
  white-space: nowrap;
  word-wrap: normal;
  direction: ltr;
  vertical-align: middle;
  -webkit-font-smoothing: antialiased;
}

.terminal-scroll::-webkit-scrollbar {
  width: 4px;
}

.terminal-scroll::-webkit-scrollbar-track {
  background: var(--surface-container-lowest);
}

.terminal-scroll::-webkit-scrollbar-thumb {
  background: var(--surface-container-highest);
  border-radius: 4px;
}

.terminal-scroll::-webkit-scrollbar-thumb:hover {
  background: var(--primary);
}

```

## src/app/page.tsx

```

import { OrchestratorApp } from "@components/orchestrator-app";

export default function Home() {
  return (
    <main className="flex-1">
      <OrchestratorApp />
    </main>
  );
}

```

```
);  
}
```

## Sistema de idiomas (ES/EN)

src/lib/i18n/dictionaries.ts

```
export type Lang = "es" | "en";  
  
export const LANGS: Lang[] = ["es", "en"];  
  
export const dictionaries = {  
  es: {  
    "nav.dashboard": "Dashboard",  
    "nav.workflows": "Workflows",  
    "nav.agents": "Agents",  
    "nav.logs": "Logs",  
    "nav.permissions": "Permisos",  
    "nav.run": "Correr Pipeline",  
    "nav.running": "Orquestando...",  
  
    "common.backToDashboard": "← Ir al Dashboard",  
    "common.language": "Idioma",  
  
    "dashboard.kicker": "Proyecto #1 · análisis de contenido",  
    "dashboard.kickerRunning": "Iniciando secuencia de análisis",  
    "dashboard.title1": "Seis agentes, un orquestador,",  
    "dashboard.title2": "cero cajas negras.",  
    "dashboard.subtitle":  
      "Extrae con evidencia, puntúa SEO, verifica claims sin inventar fuentes y recomienda acciones. El revisor  
audita a sus pares. Si la confianza baja, entra un humano.",  
    "dashboard.briefLabel": "Brief o artículo",  
    "dashboard.restoreSample": "Restaurar ejemplo",  
    "dashboard.urlLabel": "URL de Origen",  
    "dashboard.urlPlaceholder": "https://ejemplo.com/doc-tecnico...",  
    "dashboard.contentLabel": "Contexto del Documento",  
    "dashboard.contentPlaceholder":  
      "Pegue el contenido raw o el JSON estructurado aquí...",  
    "dashboard.consoleTitle": "Consola en vivo",  
    "dashboard.consoleTip":  
      "Tip: en Permisos apaga Verificación y sube el umbral HITL. El log SSE debe mostrar skip + campos con  
evidencia.",  
    "dashboard.agentMeshTitle": "Malla de Agentes",  
    "dashboard.tabTimeline": "Timeline",  
    "dashboard.tabResult": "Resultado",  
    "dashboard.emptyTimelineTitle": "Todavía no hay corrida",  
    "dashboard.emptyTimelineDesc":  
      "Pulsa Correr Pipeline para ver extracción, SEO, verificación y el checkpoint humano con confidence por  
campo.",  
    "dashboard.emptyResultTitle": "Sin paquete consolidado",  
    "dashboard.emptyResultDesc":  
      "El orquestador llena esta vista cuando termina: campos, claims, reviews y firma humana.",  
    "dashboard.sheetTitle": "Permisos y umbrales",  
    "dashboard.sheetDesc":  
      "Activa o apaga cada agente. Confianza mínima 0.70 y HITL 0.85 por defecto.",  
  
    "controlPanel.agentsTitle": "Agentes",  
    "controlPanel.agentsDesc":
```

```

    "Apaga un agente para saltarlo. Extractor y orquestador son requeridos para una corrida completa.",
    "controlPanel.minConfidence": "Confianza mínima",
    "controlPanel.minConfidenceHint": "Por debajo: el agente queda bloqueado.",
    "controlPanel.hitl": "Umbral HITL",
    "controlPanel.hitlHint":
        "Por debajo: requiere revisión humana. Default 0.85.",

    "agentBoard.status.idle": "En cola",
    "agentBoard.status.running": "En ejecución",
    "agentBoard.status.done": "Completado",
    "agentBoard.status.skipped": "Omitido (bypass)",
    "agentBoard.status.blocked": "Bloqueado",

    "logStream.title": "Consola en vivo",
    "logStream.waitingTitle": "SSE en espera",
    "logStream.waitingDesc":
        "Cada acción llega por Server-Sent Events: timestamp, agente, campo, confidence y evidencia.",
    "logStream.field": "campo",
    "logStream.conf": "conf",

    "resultDashboard.timeTotal": "Tiempo Total",
    "resultDashboard.activeAgents": "Agentes Activos",
    "resultDashboard.stepsCompleted": "Pasos Completados",
    "resultDashboard.avgConfidence": "Confianza (Avg)",
    "resultDashboard.tableAgent": "Agente",
    "resultDashboard.tableDecision": "Decisión",
    "resultDashboard.tableConf": "Conf.",
    "resultDashboard.tableSummary": "Resumen",
    "resultDashboard.chipMin": "min",
    "resultDashboard.chipHitl": "HITL",
    "resultDashboard.exportJson": "JSON",
    "resultDashboard.exportPdf": "PDF",

    "resultPanel.source": "Fuente",
    "resultPanel.hitlTitle": "Intervención Humana Requerida (HITL)",
    "resultPanel.hitlDesc":
        "{count} campo(s) por debajo del umbral. Confirma antes de tratar este paquete como publicable.",
    "resultPanel.sign": "Firmar paquete",
    "resultPanel.signed": "Paquete firmado",
    "resultPanel.missingConfirmations": "Faltan {count} confirmaciones",
    "resultPanel.approve": "Aprobar este valor manualmente",
    "resultPanel.evidence": "Evidencia",
    "resultPanel.fieldsTitle": "Campos Extraídos",
    "resultPanel.claimsTitle": "Claims & Notas",
    "resultPanel.noClaims":
        "Sin claims: verificación deshabilitada o sin permisos.",
    "resultPanel.recommendationsTitle": "Recomendaciones del Agente",
    "resultPanel.reviewsTitle": "Revisión entre agentes",
    "resultPanel.confidenceLabel": "confianza",

    "decision.aprobado": "aprobado",
    "decision.requiere revisión": "requiere revisión",
    "decision.bloqueado": "bloqueado",

    "footer.version": "Helix Orchestrator v2.4.0",
    "footer.docs": "Documentation",
    "footer.support": "Support",
    "footer.status": "System Status",

    "permission.read_input": "Leer input",
    "permission.fetch_url": "Fetch de URL",
    "permission.write_structured": "Escribir campos",

```



```

"permission.analyze_seo": "Analizar SEO",
"permission.verify_claims": "Verificar hechos",
"permission.write_recommendations": "Escribir recomendaciones",
"permission.peer_review": "Revisión entre agentes",
"permission.request_human": "Escalar a humano",

"agent.orchestrator.name": "Orquestador",
"agent.orchestrator.role": "Clasifica el input, arma el flujo y consolida el resultado.",
"agent.orchestrator.detail":
  "Clasifica el input como url, article o copy, decide si autoriza el fetch de una URL, ordena el grafo
(extractor → SEO || verificación → revisor → recomendador → revisor → consolidar) y calcula la confianza global
al final de la corrida.",
"agent.extractor.name": "Extractor",
"agent.extractor.role": "Campos con evidencia y confidence score.",
"agent.extractor.detail":
  "Estructura título, audiencia, keywords, tono y extensión a partir del texto normalizado. Cada campo trae
su propio confidence score y una cita textual como evidencia.",
"agent.seo.name": "Calidad / SEO",
"agent.seo.role": "Legibilidad, keywords y huecos de contenido.",
"agent.seo.detail":
  "Corre en paralelo con Verificación. Calcula legibilidad, densidad de palabras y si la keyword principal
aparece en el lead; devuelve además una meta description sugerida.",
"agent.factcheck.name": "Verificación",
"agent.factcheck.role": "Claims soportadas o sin fuente. No inventa.",
"agent.factcheck.detail":
  "Separa oraciones con cifras o afirmaciones fuertes y las marca como supported (citan fuente), unverified
(sin fuente) o conflicted. Nunca inventa una fuente que no existe en el texto.",
"agent.recommender.name": "Recomendaciones",
"agent.recommender.role": "Acciones priorizadas sobre métricas reales.",
"agent.recommender.detail":
  "Lee los outputs de extractor, SEO y verificación –nunca calcula sus propios números– y prioriza hasta 5
acciones accionables en alta, media o baja prioridad.",
"agent.reviewer.name": "Revisor cruzado",
"agent.reviewer.role": "Un agente revisa a otro: aprueba, corrige o bloquea.",
"agent.reviewer.detail":
  "Corre dos veces: primero sobre extractor, SEO y verificación; luego sobre el recomendador. Emite
approve, revise o block por cada agente que evalúa (agent-reviewing-agent).",

"agentsPage.kicker": "Catálogo",
"agentsPage.title": "Agentes del sistema",
"agentsPage.subtitle":
  "Seis agentes especializados, cada uno con permisos granulares propios. Actívalos, desactívalos o ajusta
los umbrales desde el panel de Permisos en el Dashboard.",
"agentsPage.defaultPerms": "Permisos por defecto",
"agentsPage.noDefaultPerms": "Sin permisos por defecto",

"workflowsPage.kicker": "Definición del pipeline",
"workflowsPage.title": "Content Intelligence Pipeline",
"workflowsPage.subtitle":
  "Un único workflow, siete pasos, con una rama en paralelo y una revisión cruzada que corre dos veces. Los
permisos y umbrales (confianza mínima 0.70, HITL 0.85 por defecto) se configuran en el Dashboard y afectan
directamente qué pasos se saltan o bloquean.",
"workflowsPage.step1.title": "1. Clasificación",
"workflowsPage.step1.detail":
  "El orquestador decide si el input es url, article o copy (según longitud del texto o si llegó una URL) y
arma el orden del grafo según ese tipo.",
"workflowsPage.step1.meta": "Orquestador",
"workflowsPage.step2.title": "2. Extracción",
"workflowsPage.step2.detail":
  "El extractor construye 5 campos (título, audiencia, keywords, tono, extensión), cada uno con su propio
confidence score y una cita como evidencia.",
"workflowsPage.step2.meta": "Extractor",

```

```

    "workflowsPage.step3.title": "3. SEO y Verificación en paralelo",
    "workflowsPage.step3.detail":
      "SEO calcula legibilidad y densidad de keywords; Verificación separa afirmaciones en supported /
      unverified / conflicted. Corren al mismo tiempo y cada uno respeta sus propios permisos.",
    "workflowsPage.step3.meta": "Calidad / SEO - Verificación",
    "workflowsPage.step4.title": "4. Revisión cruzada (pase 1)",
    "workflowsPage.step4.detail":
      "El revisor evalúa extractor, SEO y verificación. Emite approve, revise o block por cada uno - agent-
      reviewing-agent.",
    "workflowsPage.step4.meta": "Revisor cruzado",
    "workflowsPage.step5.title": "5. Recomendaciones",
    "workflowsPage.step5.detail":
      "El recomendador lee los outputs anteriores (nunca calcula sus propios números) y prioriza hasta 5
      acciones en alta / media / baja prioridad.",
    "workflowsPage.step5.meta": "Recomendaciones",
    "workflowsPage.step6.title": "6. Revisión cruzada (pase 2)",
    "workflowsPage.step6.detail":
      "El revisor vuelve a correr, esta vez sobre el recomendador, y actualiza su resumen de decisiones.",
    "workflowsPage.step6.meta": "Revisor cruzado",
    "workflowsPage.step7.title": "7. Consolidación",
    "workflowsPage.step7.detail":
      "El orquestador promedia la confianza de los agentes que terminaron, decide si se requiere HITL (campo
      por debajo del umbral o confianza global baja) y narra el resultado con Claude si hay API key configurada.",
    "workflowsPage.step7.meta": "Orquestador",
    "workflowsPage.inputKindsTitle": "Tipos de input y cómo cambian el flujo",
    "workflowsPage.kind.url.label": "url",
    "workflowsPage.kind.url.detail":
      "Llega una URL pública. El orquestador hace fetch (si tiene el permiso fetch_url) y convierte el HTML a
      texto antes de seguir.",
    "workflowsPage.kind.article.label": "article",
    "workflowsPage.kind.article.detail":
      "Texto de 280+ caracteres sin URL. Flujo completo: SEO evalúa profundidad y estructura de H2/H3
      implícita.",
    "workflowsPage.kind.copy.label": "copy",
    "workflowsPage.kind.copy.detail":
      "Texto corto (<280 caracteres). La audiencia se infiere como 'visitante de landing' y las recomendaciones
      priorizan el hook inicial.",
    "workflowsPage.thresholdsTitle": "Umbrales que gobiernan las decisiones",
    "workflowsPage.minConfTitle": "Confianza mínima - 0.70",
    "workflowsPage.minConfDetail":
      "Si un agente termina por debajo de este valor, su decisión pasa a \"bloqueado\" en el dashboard de
      resultados.",
    "workflowsPage.hitlTitle": "Umbral HITL - 0.85",
    "workflowsPage.hitlDetail":
      "Por debajo de este valor, un campo queda marcado needsHuman y el paquete requiere firma humana antes de
      publicarse.",

    "logsPage.kicker": "Auditoría",
    "logsPage.title": "Logs persistidos",
    "logsPage.subtitle":
      "Cada corrida que pasa por Supabase queda aquí, agrupada por run_id, con timestamp, agente, campo,
      confidence y evidencia - exactamente lo que se ve en la consola en vivo del Dashboard, pero persistido.",
    "logsClient.loading": "Cargando...",
    "logsClient.disabledTitle": "Persistencia deshabilitada",
    "logsClient.disabledDesc":
      "Configura NEXT_PUBLIC_SUPABASE_URL y SUPABASE_SERVICE_ROLE_KEY para guardar cada corrida. Mientras
      tanto, la consola en vivo del Dashboard sigue funcionando por SSE durante una corrida activa.",
    "logsClient.noRunsTitle": "Sin corridas registradas",
    "logsClient.noRunsDesc":
      "Corre el pipeline en el Dashboard; cada log queda persistido aquí automáticamente por run_id.",
    "logsClient.refresh": "Actualizar",
  },

```

```

en: {
  "nav.dashboard": "Dashboard",
  "nav.workflows": "Workflows",
  "nav.agents": "Agents",
  "nav.logs": "Logs",
  "nav.permissions": "Permissions",
  "nav.run": "Run Pipeline",
  "nav.running": "Orchestrating...",

  "common.backToDashboard": "← Back to Dashboard",
  "common.language": "Language",

  "dashboard.kicker": "Project #1 · content analysis",
  "dashboard.kickerRunning": "Starting analysis sequence",
  "dashboard.title1": "Six agents, one orchestrator,",
  "dashboard.title2": "zero black boxes.",
  "dashboard.subtitle":
    "Extracts with evidence, scores SEO, verifies claims without inventing sources, and recommends actions.
The reviewer audits its peers. If confidence drops, a human steps in.",
  "dashboard.briefLabel": "Brief or article",
  "dashboard.restoreSample": "Restore sample",
  "dashboard.urlLabel": "Source URL",
  "dashboard.urlPlaceholder": "https://example.com/tech-doc...",
  "dashboard.contentLabel": "Document content",
  "dashboard.contentPlaceholder":
    "Paste raw content or structured JSON here...",
  "dashboard.consoleTitle": "Live console",
  "dashboard.consoleTip":
    "Tip: in Permissions, turn off Fact-check and raise the HITL threshold. The SSE log should show a skip +
fields with evidence.",
  "dashboard.agentMeshTitle": "Agent Mesh",
  "dashboard.tabTimeline": "Timeline",
  "dashboard.tabResult": "Result",
  "dashboard.emptyTimelineTitle": "No run yet",
  "dashboard.emptyTimelineDesc":
    "Press Run Pipeline to see extraction, SEO, fact-check, and the human checkpoint with per-field
confidence.",
  "dashboard.emptyResultTitle": "No consolidated package",
  "dashboard.emptyResultDesc":
    "The orchestrator fills this view when it finishes: fields, claims, reviews, and human sign-off.",
  "dashboard.sheetTitle": "Permissions & thresholds",
  "dashboard.sheetDesc":
    "Turn each agent on or off. Minimum confidence 0.70 and HITL 0.85 by default.",

  "controlPanel.agentsTitle": "Agents",
  "controlPanel.agentsDesc":
    "Turn off an agent to skip it. Extractor and orchestrator are required for a full run.",
  "controlPanel.minConfidence": "Minimum confidence",
  "controlPanel.minConfidenceHint": "Below this: the agent is blocked.",
  "controlPanel.hitl": "HITL threshold",
  "controlPanel.hitlHint":
    "Below this: requires human review. Default 0.85.",

  "agentBoard.status.idle": "Queued",
  "agentBoard.status.running": "Running",
  "agentBoard.status.done": "Completed",
  "agentBoard.status.skipped": "Skipped (bypass)",
  "agentBoard.status.blocked": "Blocked",

  "logStream.title": "Live console",
  "logStream.waitingTitle": "SSE standing by",
  "logStream.waitingDesc":

```

```

    "Every action arrives via Server-Sent Events: timestamp, agent, field, confidence, and evidence.",
    "logStream.field": "field",
    "logStream.conf": "conf",

    "resultDashboard.timeTotal": "Total Time",
    "resultDashboard.activeAgents": "Active Agents",
    "resultDashboard.stepsCompleted": "Steps Completed",
    "resultDashboard.avgConfidence": "Confidence (Avg)",
    "resultDashboard.tableAgent": "Agent",
    "resultDashboard.tableDecision": "Decision",
    "resultDashboard.tableConf": "Conf.",
    "resultDashboard.tableSummary": "Summary",
    "resultDashboard.chipMin": "min",
    "resultDashboard.chipHitl": "HITL",
    "resultDashboard.exportJson": "JSON",
    "resultDashboard.exportPdf": "PDF",

    "resultPanel.source": "Source",
    "resultPanel.hitlTitle": "Human Intervention Required (HITL)",
    "resultPanel.hitlDesc":
        "{count} field(s) below threshold. Confirm before treating this package as publishable.",
    "resultPanel.sign": "Sign package",
    "resultPanel.signed": "Package signed",
    "resultPanel.missingConfirmations": "{count} confirmations missing",
    "resultPanel.approve": "Approve this value manually",
    "resultPanel.evidence": "Evidence",
    "resultPanel.fieldsTitle": "Extracted Fields",
    "resultPanel.claimsTitle": "Claims & Notes",
    "resultPanel.noClaims":
        "No claims: fact-check disabled or missing permissions.",
    "resultPanel.recommendationsTitle": "Agent Recommendations",
    "resultPanel.reviewsTitle": "Cross-agent review",
    "resultPanel.confidenceLabel": "confidence",

    "decision.aprobado": "approved",
    "decision.requiere revisión": "needs review",
    "decision.bloqueado": "blocked",

    "footer.version": "Helix Orchestrator v2.4.0",
    "footer.docs": "Documentation",
    "footer.support": "Support",
    "footer.status": "System Status",

    "permission.read_input": "Read input",
    "permission.fetch_url": "Fetch URL",
    "permission.write_structured": "Write fields",
    "permission.analyze_seo": "Analyze SEO",
    "permission.verify_claims": "Verify claims",
    "permission.write_recommendations": "Write recommendations",
    "permission.peer_review": "Cross-agent review",
    "permission.request_human": "Escalate to human",

    "agent.orchestrator.name": "Orchestrator",
    "agent.orchestrator.role": "Classifies the input, builds the flow, and consolidates the result.",
    "agent.orchestrator.detail":
        "Classifies the input as url, article, or copy, decides whether to authorize fetching a URL, orders the
        graph (extractor → SEO || fact-check → reviewer → recommender → reviewer → consolidate), and computes overall
        confidence at the end of the run.",
    "agent.extractor.name": "Extractor",
    "agent.extractor.role": "Fields with evidence and confidence score.",
    "agent.extractor.detail":
        "Structures title, audience, keywords, tone, and length from the normalized text. Each field carries its

```

```

own confidence score and a textual quote as evidence.",
"agent.seo.name": "Quality / SEO",
"agent.seo.role": "Readability, keywords, and content gaps.",
"agent.seo.detail":
  "Runs in parallel with Fact-check. Computes readability, word density, and whether the main keyword
  appears in the lead; also returns a suggested meta description.",
"agent.factcheck.name": "Fact-check",
"agent.factcheck.role": "Claims supported or unsourced. Never invents.",
"agent.factcheck.detail":
  "Splits sentences with numbers or strong claims and marks them as supported (cite a source), unverified
  (no source), or conflicted. Never invents a source that isn't in the text.",
"agent.recommender.name": "Recommender",
"agent.recommender.role": "Prioritized actions based on real metrics.",
"agent.recommender.detail":
  "Reads the outputs from extractor, SEO, and fact-check –never computes its own numbers– and prioritizes
  up to 5 actionable items as high, medium, or low priority.",
"agent.reviewer.name": "Cross Reviewer",
"agent.reviewer.role": "One agent reviews another: approves, revises, or blocks.",
"agent.reviewer.detail":
  "Runs twice: first over extractor, SEO, and fact-check; then over the recommender. Issues approve,
  revise, or block for each agent it evaluates (agent-reviewing-agent).",

"agentsPage.kicker": "Catalog",
"agentsPage.title": "System agents",
"agentsPage.subtitle":
  "Six specialized agents, each with its own granular permissions. Turn them on, off, or adjust thresholds
  from the Permissions panel on the Dashboard.",
"agentsPage.defaultPerms": "Default permissions",
"agentsPage.noDefaultPerms": "No default permissions",

"workflowsPage.kicker": "Pipeline definition",
"workflowsPage.title": "Content Intelligence Pipeline",
"workflowsPage.subtitle":
  "A single workflow, seven steps, with one parallel branch and a cross-review that runs twice. Permissions
  and thresholds (minimum confidence 0.70, HITL 0.85 by default) are configured on the Dashboard and directly
  affect which steps are skipped or blocked.",
"workflowsPage.step1.title": "1. Classification",
"workflowsPage.step1.detail":
  "The orchestrator decides whether the input is url, article, or copy (based on text length or whether a
  URL arrived) and orders the graph based on that type.",
"workflowsPage.step1.meta": "Orchestrator",
"workflowsPage.step2.title": "2. Extraction",
"workflowsPage.step2.detail":
  "The extractor builds 5 fields (title, audience, keywords, tone, length), each with its own confidence
  score and a quote as evidence.",
"workflowsPage.step2.meta": "Extractor",
"workflowsPage.step3.title": "3. SEO and Fact-check in parallel",
"workflowsPage.step3.detail":
  "SEO computes readability and keyword density; Fact-check splits claims into supported / unverified /
  conflicted. They run at the same time and each respects its own permissions.",
"workflowsPage.step3.meta": "Quality / SEO · Fact-check",
"workflowsPage.step4.title": "4. Cross review (pass 1)",
"workflowsPage.step4.detail":
  "The reviewer evaluates extractor, SEO, and fact-check. It issues approve, revise, or block for each –
  agent-reviewing-agent.",
"workflowsPage.step4.meta": "Cross Reviewer",
"workflowsPage.step5.title": "5. Recommendations",
"workflowsPage.step5.detail":
  "The recommender reads the previous outputs (never computes its own numbers) and prioritizes up to 5
  actions as high / medium / low priority.",
"workflowsPage.step5.meta": "Recommender",
"workflowsPage.step6.title": "6. Cross review (pass 2)",

```

```

    "workflowsPage.step6.detail":
      "The reviewer runs again, this time over the recommender, and updates its decision summary.",
    "workflowsPage.step6.meta": "Cross Reviewer",
    "workflowsPage.step7.title": "7. Consolidation",
    "workflowsPage.step7.detail":
      "The orchestrator averages the confidence of the agents that finished, decides whether HITL is required
(a field below threshold or low overall confidence), and narrates the result with Claude if an API key is
configured.",
    "workflowsPage.step7.meta": "Orchestrator",
    "workflowsPage.inputKindsTitle": "Input types and how they change the flow",
    "workflowsPage.kind.url.label": "url",
    "workflowsPage.kind.url.detail":
      "A public URL arrives. The orchestrator fetches it (if it has the fetch_url permission) and converts the
HTML to text before continuing.",
    "workflowsPage.kind.article.label": "article",
    "workflowsPage.kind.article.detail":
      "280+ character text with no URL. Full flow: SEO evaluates depth and implied H2/H3 structure.",
    "workflowsPage.kind.copy.label": "copy",
    "workflowsPage.kind.copy.detail":
      "Short text (<280 characters). Audience is inferred as 'landing page visitor' and recommendations
prioritize the opening hook.",
    "workflowsPage.thresholdsTitle": "Thresholds that govern decisions",
    "workflowsPage.minConfTitle": "Minimum confidence – 0.70",
    "workflowsPage.minConfDetail":
      "If an agent finishes below this value, its decision becomes \"blocked\" in the results dashboard.",
    "workflowsPage.hitlTitle": "HITL threshold – 0.85",
    "workflowsPage.hitlDetail":
      "Below this value, a field is flagged needsHuman and the package requires human sign-off before
publishing.",

    "logsPage.kicker": "Audit",
    "logsPage.title": "Persisted logs",
    "logsPage.subtitle":
      "Every run that passes through Supabase lands here, grouped by run_id, with timestamp, agent, field,
confidence, and evidence – exactly what you see in the Dashboard's live console, but persisted.",
    "logsClient.loading": "Loading...",
    "logsClient.disabledTitle": "Persistence disabled",
    "logsClient.disabledDesc":
      "Configure NEXT_PUBLIC_SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY to save every run. In the meantime, the
Dashboard's live console keeps working over SSE during an active run.",
    "logsClient.noRunsTitle": "No runs recorded",
    "logsClient.noRunsDesc":
      "Run the pipeline on the Dashboard; every log is automatically persisted here by run_id.",
    "logsClient.refresh": "Refresh",
  },
} as const;

export type TranslationKey = keyof typeof dictionaries.es;

```

## src/lib/i18n/language-provider.tsx

```

"use client";

import {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useState,

```

```

} from "react";
import { dictionaries, type Lang, type TranslationKey } from "../dictionaries";

const STORAGE_KEY = "helix.lang";

type LanguageContextValue = {
  lang: Lang;
  setLang: (lang: Lang) => void;
  toggle: () => void;
  t: (key: TranslationKey, vars?: Record<string, string | number>) => string;
};

const LanguageContext = createContext<LanguageContextValue | null>(null);

function interpolate(
  template: string,
  vars?: Record<string, string | number>
): string {
  if (!vars) return template;
  return template.replace(/\{(\w+)\}/g, (match, key) =>
    key in vars ? String(vars[key]) : match
  );
}

const DEFAULT_LANG: Lang = "es";

export function LanguageProvider({ children }: { children: React.ReactNode }) {
  // Always start with the server-rendered default so the first client
  // render matches SSR output exactly (no hydration mismatch). The
  // persisted preference is applied right after mount, as a normal
  // post-hydration update instead of during the hydration pass itself.
  const [lang, setLangState] = useState<Lang>(DEFAULT_LANG);

  useEffect(() => {
    const stored = window.localStorage.getItem(STORAGE_KEY);
    const next = stored === "en" ? "en" : stored === "es" ? "es" : null;
    if (next && next !== DEFAULT_LANG) {
      queueMicrotask(() => setLangState(next));
    }
  }, []);

  useEffect(() => {
    document.documentElement.lang = lang;
  }, [lang]);

  const setLang = useCallback((next: Lang) => {
    setLangState(next);
    window.localStorage.setItem(STORAGE_KEY, next);
  }, []);

  const toggle = useCallback(() => {
    setLang(lang === "es" ? "en" : "es");
  }, [lang, setLang]);

  const t = useCallback(
    (key: TranslationKey, vars?: Record<string, string | number>) => {
      const dict = dictionaries[lang];
      const value = (dict as Record<string, string>)[key] ?? key;
      return interpolate(value, vars);
    },
    [lang]
  );
}

```

```

const value = useMemo(
  () => ({ lang, setLang, toggle, t }),
  [lang, setLang, toggle, t]
);

return (
  <LanguageContext.Provider value={value}>
    {children}
  </LanguageContext.Provider>
);
}

export function useLanguage(): LanguageContextValue {
  const ctx = useContext(LanguageContext);
  if (!ctx) {
    throw new Error("useLanguage must be used within a LanguageProvider");
  }
  return ctx;
}

```

### src/components/language-toggle.tsx

```

"use client";

import { useLanguage } from "@/lib/i18n/language-provider";
import { cn } from "@/lib/utils";

export function LanguageToggle() {
  const { lang, setLang, t } = useLanguage();

  return (
    <div
      role="group"
      aria-label={t("common.language")}
      className="border-outline-variant/40 flex items-center rounded-full border p-0.5 text-[11px]"
    >
      {[["es", "en"] as const].map((option) => (
        <button
          key={option}
          type="button"
          onClick={() => setLang(option)}
          aria-pressed={lang === option}
          className={cn(
            "font-label-sm rounded-full px-2.5 py-1 uppercase tracking-wide transition-colors",
            lang === option
              ? "bg-primary text-on-primary"
              : "text-on-surface-variant hover:text-on-surface"
          )}
        >
          {option}
        </button>
      ))}
    </div>
  );
}

```



## Nav, footer e íconos compartidos

src/components/top-nav.tsx

```
"use client";

import Link from "next/link";
import { Icon } from "@components/icon";
import { LanguageToggle } from "@components/language-toggle";
import { useLanguage } from "@lib/i18n/language-provider";
import { cn } from "@lib/utils";

export type NavKey = "dashboard" | "workflows" | "agents" | "logs";

const NAV_ITEMS: { href: string; key: NavKey; labelKey: "nav.dashboard" | "nav.workflows" | "nav.agents" | "nav.logs" }[] = [
  { href: "/", key: "dashboard", labelKey: "nav.dashboard" },
  { href: "/workflows", key: "workflows", labelKey: "nav.workflows" },
  { href: "/agents", key: "agents", labelKey: "nav.agents" },
  { href: "/logs", key: "logs", labelKey: "nav.logs" },
];

export function TopNav({
  active,
  pulse = "ok",
  actions,
}: {
  active: NavKey;
  pulse?: "busy" | "ok";
  actions?: React.ReactNode;
}) {
  const { t } = useLanguage();

  return (
    <nav className="border-outline-variant/30 bg-surface/80 fixed top-0 z-50 w-full border-b shadow-sm backdrop-blur-xl">
      <div className="mx-auto flex h-16 max-w-[1440px] items-center justify-between px-4 sm:px-6">
        <Link href="/" className="flex items-center gap-2">
          <span className="text-primary flex items-center text-[24px] font-bold leading-8 drop-shadow-[0_0_8px_rgba(76,215,246,0.15)]">
            <Icon name="hub" filled className="mr-2 text-[24px]" />
            HX
          </span>
          <div className="bg-outline-variant mx-2 h-4 w-px" />
          <span className="text-on-surface text-[16px] font-medium">
            Helix Orchestrator
          </span>
          <span className="relative ml-2 flex h-2 w-2">
            <span
              className={cn(
                "absolute inline-flex h-full w-full rounded-full opacity-75",
                pulse === "busy" ? "animate-ping bg-primary" : "bg-tertiary"
              )}
            />
            <span
              className={cn(
                "relative inline-flex h-2 w-2 rounded-full",
                pulse === "busy" ? "bg-primary" : "bg-tertiary"
              )}
            />
          </span>
        </Link>
      </div>
    </nav>
  );
}
```

```

    </span>
  </Link>

  <div className="hidden items-center gap-6 md:flex">
    {NAV_ITEMS.map((item) => (
      <Link
        key={item.key}
        href={item.href}
        className={cn(
          "font-label-sm border-b-2 pb-1 text-[12px] uppercase tracking-wider transition-colors",
          active === item.key
            ? "text-primary border-primary"
            : "text-on-surface-variant hover:text-primary border-transparent"
        )}
      >
        {t(item.labelKey)}
      </Link>
    ))}
  </div>

  <div className="flex items-center gap-3">
    <LanguageToggle />
    <Icon
      name="notifications"
      className="text-on-surface-variant hidden text-[20px] sm:inline"
    />
    <div className="bg-outline-variant mx-1 hidden h-6 w-px sm:block" />
    {actions}
  </div>
</div>
</nav>
);
}

```

## src/components/page-footer.tsx

```

"use client";

import { useLanguage } from "@lib/i18n/language-provider";

export function PageFooter() {
  const { t } = useLanguage();

  return (
    <footer className="border-outline-variant/20 bg-surface-dim mt-8 w-full border-t py-4">
      <div className="mx-auto flex max-w-[1440px] flex-col items-center justify-between gap-2 px-4 sm:flex-row sm:px-6">
        <p className="text-on-surface-variant font-label-sm text-[12px]">
          {t("footer.version")}
        </p>
        <div className="flex gap-4">
          <span className="text-on-surface-variant hover:text-tertiary cursor-default text-[13px] underline">
            {t("footer.docs")}
          </span>
          <span className="text-on-surface-variant hover:text-tertiary cursor-default text-[13px] underline">
            {t("footer.support")}
          </span>
          <span className="text-on-surface-variant hover:text-tertiary cursor-default text-[13px] underline">
            {t("footer.status")}
          </span>
        </div>
      </div>
    </footer>
  );
}

```

```

        </div>
      </div>
    </footer>
  );
}

```

## src/components/icon.tsx

```

import { cn } from "@lib/utils";

export function Icon({
  name,
  filled = false,
  className,
}: {
  name: string;
  filled?: boolean;
  className?: string;
}) {
  return (
    <span
      className={cn("material-symbols-outlined select-none", className)}
      style={filled ? { fontVariationSettings: "'FILL' 1" } : undefined}
      aria-hidden="true"
    >
      {name}
    </span>
  );
}

```

# Dashboard rediseñado

## src/components/orchestrator-app.tsx

```

"use client";

import { useState } from "react";
import { AgentBoard } from "@components/agent-board";
import { ControlPanel } from "@components/control-panel";
import { ExportButtons } from "@components/result-dashboard";
import { Icon } from "@components/icon";
import { LogStream } from "@components/log-stream";
import { PageFooter } from "@components/page-footer";
import { ResultPanel } from "@components/result-panel";
import { TopNav } from "@components/top-nav";
import {
  Sheet,
  SheetContent,
  SheetDescription,
  SheetHeader,
  SheetTitle,
  SheetTrigger,
} from "@components/ui/sheet";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
import {
  DEFAULT_ENABLED_AGENTS,
  DEFAULT_HITL_THRESHOLD,

```

```

    DEFAULT_MIN_CONFIDENCE,
  } from "@lib/config";
import { useLanguage } from "@lib/i18n/language-provider";
import { AGENT_CATALOG, DEFAULT_PERMISSIONS } from "@lib/permissions";
import { SAMPLE_ARTICLE } from "@lib/sample";
import { looksLikeUrl } from "@lib/text";
import type {
  AgentEnabledMap,
  AgentRun,
  PermissionMap,
  PipelineLog,
  PipelineResult,
  StreamEvent,
} from "@lib/types";
import { cn } from "@lib/utils";

export function OrchestratorApp() {
  const { t } = useLanguage();
  const [text, setText] = useState(SAMPLE_ARTICLE);
  const [url, setUrl] = useState("");
  const [permissions] = useState<PermissionMap>(DEFAULT_PERMISSIONS);
  const [enabledAgents, setEnabledAgents] =
    useState<AgentEnabledMap>(DEFAULT_ENABLED_AGENTS);
  const [minConfidence, setMinConfidence] = useState(DEFAULT_MIN_CONFIDENCE);
  const [hitlThreshold, setHitlThreshold] = useState(DEFAULT_HITL_THRESHOLD);
  const [logs, setLogs] = useState<PipelineLog[]>([]);
  const [runs, setRuns] = useState<AgentRun[]>([]);
  const [result, setResult] = useState<PipelineResult | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [running, setRunning] = useState(false);
  const [tab, setTab] = useState("timeline");

  async function analyze() {
    setRunning(true);
    setError(null);
    setLogs([]);
    setRuns([]);
    setResult(null);
    setTab("timeline");

    const trimmedUrl = url.trim() || (looksLikeUrl(text) ? text.trim() : "");
    const payload = {
      ...(trimmedUrl
        ? { text: "", url: trimmedUrl }
        : { text, url: undefined }),
      permissions,
      enabledAgents,
      minConfidence,
      hitlThreshold,
    };

    try {
      const res = await fetch("/api/analyze", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload),
      });
      if (!res.ok || !res.body) {
        throw new Error(`HTTP ${res.status}`);
      }

      const reader = res.body.getReader();

```

```

const decoder = new TextDecoder();
let buffer = "";

while (true) {
  const { value, done } = await reader.read();
  if (done) break;
  buffer += decoder.decode(value, { stream: true });
  const chunks = buffer.split("\n\n");
  buffer = chunks.pop() ?? "";
  for (const chunk of chunks) {
    const line = chunk
      .split("\n")
      .filter((l) => l.startsWith("data:"))
      .map((l) => l.slice(5).trim())
      .join("");
    if (!line) continue;
    const event = JSON.parse(line) as StreamEvent;
    if (event.type === "log") {
      setLogs((prev) => [...prev, event.log]);
    } else if (event.type === "agent") {
      setRuns((prev) => {
        const next = prev.filter((r) => r.agent !== event.run.agent);
        return [...next, event.run];
      });
    } else if (event.type === "result") {
      setResult(event.result);
      setTab("result");
    } else if (event.type === "error") {
      setError(event.message);
    }
  }
}

} catch (err) {
  setError(err instanceof Error ? err.message : String(err));
} finally {
  setRunning(false);
}
}

const runFor = (id: string) => runs.find((r) => r.agent === id);

return (
  <div className="text-on-surface font-body-md flex min-h-full flex-col">
    <TopNav
      active="dashboard"
      pulse={running ? "busy" : "ok"}
      actions={
        <
          <Sheet>
            <SheetTrigger className="border-outline-variant bg-surface-container-low text-on-surface
hover:border-primary hover:text-primary font-label-sm rounded border px-3 py-1.5 text-[12px] transition-
colors">
              <span className="flex items-center gap-1.5">
                <Icon name="tune" className="text-[16px]" />
                {t("nav.permissions")}
              </span>
            </SheetTrigger>
            <SheetContent
              side="right"
              className="w-full overflow-y-auto data-[side=right]:sm:max-w-md"
            >
              <SheetHeader>

```

```

        <SheetTitle>{t("dashboard.sheetTitle")}</SheetTitle>
        <SheetDescription>
            {t("dashboard.sheetDesc")}
        </SheetDescription>
    </SheetHeader>
    <div className="px-4 pb-6">
        <ControlPanel
            enabled={enabledAgents}
            onEnabled={setEnabledAgents}
            minConfidence={minConfidence}
            hitlThreshold={hitlThreshold}
            onMinConfidence={setMinConfidence}
            onHitlThreshold={setHitlThreshold}
        />
    </div>
</SheetContent>
</Sheet>
<button
    type="button"
    onClick={analyze}
    disabled={running}
    className="bg-primary text-on-primary hover:glow-primary font-label-sm flex items-center gap-2
rounded px-3 py-1.5 text-[12px] font-bold transition-all disabled:opacity-50"
>
    <Icon name="play_arrow" filled className="text-[16px]" />
    {running ? t("nav.running") : t("nav.run")}
</button>
</>
}
/>

<main className="mx-auto flex w-full max-w-[1440px] flex-1 flex-col gap-8 px-4 py-8 pt-24 sm:px-6">
    {/* Hero */}
    <section className="flex flex-col gap-4">
        <div>
            <p className="text-primary glow-text-primary mb-2 text-[12px] uppercase tracking-widest">
                {running ? t("dashboard.kickerRunning") : t("dashboard.kicker")}
            </p>
            <h1 className="from-primary to-secondary bg-gradient-to-r bg-clip-text text-[32px] font-bold
leading-10 text-transparent sm:text-[40px] sm:leading-[48px]">
                {t("dashboard.title1")}
            <br />
                {t("dashboard.title2")}
            </h1>
        </div>

        {/* Stepper */}
        <div className="mt-2 flex w-full items-center gap-1 overflow-x-auto py-2">
            {AGENT_CATALOG.map((agent, i) => {
                const run = runFor(agent.id);
                const status = run?.status ?? "idle";
                const isActive = status === "running" || status === "done";
                return (
                    <div key={agent.id} className="flex items-center">
                        <div
                            className={cn(
                                "relative z-10 flex items-center gap-2 rounded-full px-3 py-1.5 text-[12px]",
                                status === "running" &&
                                    "glow-primary border-primary bg-primary/20 text-primary border",
                                status === "done" &&
                                    "border-tertiary bg-surface-container-high text-tertiary border",
                                (status === "idle" || status === "skipped" || status === "blocked") &&

```

```

        "border-outline-variant bg-surface-container-high text-on-surface-variant border
opacity-60"
    })
  >
    <Icon name={agent.icon} className="text-[14px]" />
    {agent.short}
  </div>
  {i < AGENT_CATALOG.length - 1 ? (
    <div
      className={cn(
        "-ml-1 h-px w-8",
        isActive ? "bg-primary/60" : "bg-outline-variant/40"
      )}
    />
  ) : null}
</div>
);
}}
</div>
</section>

{/* Input + Console */}
<section className="grid h-auto grid-cols-1 gap-6 lg:h-[480px] lg:grid-cols-12">
  <div className="border-surface-bright bg-surface-container relative flex flex-col gap-2 overflow-
hidden rounded-lg border p-4 lg:col-span-7">
    <div className="via-primary/30 absolute top-0 right-0 left-0 h-[1px] bg-gradient-to-r from-
transparent to-transparent" />
    <label className="text-on-surface-variant font-label-sm flex items-center gap-2 text-[12px]">
      <Icon name="link" className="text-[16px]" />
      {t("dashboard.urlLabel")}
    </label>
    <input
      value={url}
      onChange={(e) => setUrl(e.target.value)}
      placeholder={t("dashboard.urlPlaceholder")}
      type="text"
      className="border-outline-variant bg-surface-container-lowest text-on-surface focus:border-
primary focus:ring-primary font-code-md rounded border px-3 py-2 text-[14px] transition-all focus:outline-none
focus:ring-1"
    />
    <label className="text-on-surface-variant font-label-sm mt-2 flex items-center gap-2 text-[12px]">
      <Icon name="description" className="text-[16px]" />
      {t("dashboard.contentLabel")}
    </label>
    <textarea
      value={text}
      onChange={(e) => setText(e.target.value)}
      placeholder={t("dashboard.contentPlaceholder")}
      className="border-outline-variant bg-surface-container-lowest text-on-surface focus:border-
primary focus:ring-primary font-code-md min-h-[220px] flex-1 resize-none rounded border px-3 py-2.5 text-[14px]
transition-all focus:outline-none focus:ring-1"
    />
    <div className="flex items-center justify-between">
      <button
        type="button"
        onClick={() => {
          setText(SAMPLE_ARTICLE);
          setUrl("");
        }}
        className="text-on-surface-variant hover:text-primary font-label-sm text-[11px] underline"
      />
      {t("dashboard.restoreSample")}
    </div>
  </div>
  <div className="border-surface-bright bg-surface-container relative flex flex-col gap-2 overflow-
hidden rounded-lg border p-4 lg:col-span-5">
    <div className="via-primary/30 absolute top-0 right-0 left-0 h-[1px] bg-gradient-to-r from-
transparent to-transparent" />
    <label className="text-on-surface-variant font-label-sm flex items-center gap-2 text-[12px]">
      <Icon name="link" className="text-[16px]" />
      {t("dashboard.urlLabel")}
    </label>
    <input
      value={url}
      onChange={(e) => setUrl(e.target.value)}
      placeholder={t("dashboard.urlPlaceholder")}
      type="text"
      className="border-outline-variant bg-surface-container-lowest text-on-surface focus:border-
primary focus:ring-primary font-code-md rounded border px-3 py-2 text-[14px] transition-all focus:outline-none
focus:ring-1"
    />
    <label className="text-on-surface-variant font-label-sm mt-2 flex items-center gap-2 text-[12px]">
      <Icon name="description" className="text-[16px]" />
      {t("dashboard.contentLabel")}
    </label>
    <textarea
      value={text}
      onChange={(e) => setText(e.target.value)}
      placeholder={t("dashboard.contentPlaceholder")}
      className="border-outline-variant bg-surface-container-lowest text-on-surface focus:border-
primary focus:ring-primary font-code-md min-h-[220px] flex-1 resize-none rounded border px-3 py-2.5 text-[14px]
transition-all focus:outline-none focus:ring-1"
    />
    <div className="flex items-center justify-between">
      <button
        type="button"
        onClick={() => {
          setText(SAMPLE_ARTICLE);
          setUrl("");
        }}
        className="text-on-surface-variant hover:text-primary font-label-sm text-[11px] underline"
      />
      {t("dashboard.restoreSample")}
    </div>
  </div>
</section>

```

```

        </button>
        {error ? (
            <p className="text-error text-[12px]" role="alert">
                {error}
            </p>
        ) : null}
    </div>
</div>

<div className="lg:col-span-5">
    <LogStream logs={logs} running={running} />
</div>
</section>

{/* Agent Board */}
<section className="flex flex-col gap-3">
    <h2 className="text-on-surface flex items-center gap-2 text-[24px] font-semibold">
        <Icon name="group_work" className="text-primary text-[24px]" />
        {t("dashboard.agentMeshTitle")}
    </h2>
    <AgentBoard runs={runs} />
</section>

{/* Results */}
<section className="bg-surface-container-low border-outline-variant/30 mt-2 flex flex-col overflow-
hidden rounded-xl border">
    <Tabs value={tab} onValueChange={setTab}>
        <div className="border-outline-variant/30 bg-surface/50 flex items-center border-b">
            <TabsList className="bg-transparent p-0">
                <TabsTrigger value="timeline">
                    {t("dashboard.tabTimeline")}
                </TabsTrigger>
                <TabsTrigger value="result">
                    {t("dashboard.tabResult")}
                </TabsTrigger>
            </TabsList>
            <div className="flex-1" />
            {result ? (
                <div className="px-4">
                    <ExportButtons result={result} onPdfError={setError} />
                </div>
            ) : null}
        </div>

        <div className="p-4 sm:p-6">
            <TabsContent value="timeline">
                {logs.length ? (
                    <LogStream logs={logs} running={running} />
                ) : (
                    <EmptyState
                        title={t("dashboard.emptyTimelineTitle")}
                        description={t("dashboard.emptyTimelineDesc")}
                    </>
                )}
            </TabsContent>
            <TabsContent value="result">
                {result ? (
                    <ResultPanel result={result} />
                ) : (
                    <EmptyState
                        title={t("dashboard.emptyResultTitle")}
                        description={t("dashboard.emptyResultDesc")}
                    </>
                )}
            </TabsContent>
        </div>
    </Tabs>

```



```

        />
      )}
    </TabsContent>
  </div>
</Tabs>
</section>
</main>

<PageFooter />
</div>
);
}

function EmptyState({
  title,
  description,
}: {
  title: string;
  description: string;
}) {
  return (
    <div className="border-outline-variant/30 rounded-lg border border-dashed px-6 py-16 text-center">
      <p className="text-on-surface text-base font-medium">{title}</p>
      <p className="text-on-surface-variant mx-auto mt-2 max-w-md text-sm leading-6">
        {description}
      </p>
    </div>
  );
}

```

## src/components/agent-board.tsx

```

"use client";

import { Icon } from "@components/icon";
import { useLanguage } from "@lib/i18n/language-provider";
import { AGENT_CATALOG } from "@lib/permissions";
import type { AgentRun } from "@lib/types";
import { cn } from "@lib/utils";

const statusIcon: Record<AgentRun["status"], string> = {
  idle: "hourglass_empty",
  running: "sync",
  done: "check_circle",
  skipped: "fast_forward",
  blocked: "block",
};

const statusKey: Record<AgentRun["status"], `agentBoard.status.${AgentRun["status"]}`> = {
  idle: "agentBoard.status.idle",
  running: "agentBoard.status.running",
  done: "agentBoard.status.done",
  skipped: "agentBoard.status.skipped",
  blocked: "agentBoard.status.blocked",
};

export function AgentBoard({ runs }: { runs: AgentRun[] }) {
  const { t } = useLanguage();

  return (

```

```

<div className="grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3">
  {AGENT_CATALOG.map((agent) => {
    const run = runs.find((r) => r.agent === agent.id);
    const status = run?.status ?? "idle";
    const confidence = Math.round((run?.confidence ?? 0) * 100);
    const name = t(`agent.${agent.id}.name` as const);
    const role = t(`agent.${agent.id}.role` as const);

    const isRunning = status === "running";
    const isDone = status === "done";
    const isBlocked = status === "blocked";
    const isSkipped = status === "skipped";
    const isIdle = status === "idle";

    return (
      <div
        key={agent.id}
        className={cn(
          "relative overflow-hidden rounded-lg border p-4 transition-transform duration-300",
          isRunning &&
            "border-primary/40 bg-surface-container shadow-[0_4px_24px_rgba(76,215,246,0.08)] hover:-translate-y-1",
          isDone &&
            "border-tertiary/30 bg-surface-container hover:-translate-y-1",
          isBlocked &&
            "border-error/50 bg-surface-container hover:-translate-y-1",
          isSkipped &&
            "border-outline-variant/10 bg-surface-container opacity-40 grayscale",
          isIdle &&
            "border-outline-variant/30 bg-surface-container opacity-60"
        )}
      >
        {isRunning ? (
          <div className="glow-primary absolute top-0 left-0 h-[1px] w-full bg-primary" />
        ) : null}
        {isBlocked ? (
          <div className="absolute top-0 left-0 h-[1px] w-full bg-error shadow-[0_0_8px_#ffb4ab]" />
        ) : null}

        <div className="mb-2 flex items-start justify-between">
          <div className="flex items-center gap-2">
            <span
              className={cn(
                "font-label-sm rounded px-1.5 py-0.5 text-[10px] font-bold",
                isRunning && "bg-primary text-on-primary",
                isDone &&
                  "bg-surface-bright text-tertiary border-tertiary/50 border",
                isBlocked && "bg-error/20 text-error border-error/30 border",
                (isSkipped || isIdle) &&
                  "bg-surface-bright text-on-surface-variant border-outline-variant/50 border"
              )}
            >
              {agent.short}
            </span>
            <h4
              className={cn(
                "text-sm font-semibold",
                isSkipped ? "text-outline line-through" : "text-on-surface"
              )}
            >
              {name}
            </h4>
          </div>
        </div>
      </div>
    );
  })}

```

```

    </div>
    {isRunning ? (
      <span className="relative mt-1 flex h-2 w-2">
        <span className="absolute inline-flex h-full w-full animate-ping rounded-full bg-primary
opacity-75" />
        <span className="relative inline-flex h-2 w-2 rounded-full bg-primary" />
      </span>
    ) : (
      <Icon
        name={statusIcon[status]}
        className={cn(
          "text-[16px]",
          isDone && "text-tertiary",
          isBlocked && "text-error",
          (isSkipped || isIdle) && "text-on-surface-variant"
        )}
      />
    )}
  </div>

  <p
    className={cn(
      "font-body-md mb-4 text-[12px] leading-4",
      isSkipped ? "text-outline line-through" : "text-on-surface-variant"
    )}
  >
    {role}
  </p>

  <div className="mb-2 h-1 w-full overflow-hidden rounded-full bg-surface-container-highest">
    <div
      className={cn(
        "h-1 rounded-full transition-all duration-500",
        isRunning && "bg-primary",
        isDone && "bg-tertiary",
        isBlocked && "bg-error",
        (isSkipped || isIdle) && "bg-surface-bright"
      )}
      style={{ width: `${confidence}%` }}
    />
  </div>

  <div className="font-code-md flex justify-between text-[10px] text-outline">
    <span
      className={cn(
        isBlocked && "font-bold text-error",
        isDone && "text-tertiary"
      )}
    >
      {run?.summary || t(statusKey[status])}
    </span>
    <span>{run ? `${confidence}%` : "--"}</span>
  </div>
</div>
);
}}
</div>
);
}

```

```

"use client";

import { Icon } from "@components/icon";
import { Label } from "@components/ui/label";
import { Switch } from "@components/ui/switch";
import { useLanguage } from "@lib/i18n/language-provider";
import { AGENT_CATALOG } from "@lib/permissions";
import type { AgentEnabledMap, AgentId } from "@lib/types";

export function ControlPanel({
  enabled,
  onEnabled,
  minConfidence,
  hitlThreshold,
  onMinConfidence,
  onHitlThreshold,
}: {
  enabled: AgentEnabledMap;
  onEnabled: (next: AgentEnabledMap) => void;
  minConfidence: number;
  hitlThreshold: number;
  onMinConfidence: (n: number) => void;
  onHitlThreshold: (n: number) => void;
}) {
  const { t } = useLanguage();

  const toggle = (id: AgentId, value: boolean) => {
    onEnabled({ ...enabled, [id]: value });
  };

  return (
    <div className="space-y-6">
      <div>
        <p className="text-on-surface text-sm font-medium">
          {t("controlPanel.agentsTitle")}
        </p>
        <p className="text-on-surface-variant mb-3 text-xs">
          {t("controlPanel.agentsDesc")}
        </p>
        <ul className="space-y-3">
          {AGENT_CATALOG.map((agent) => (
            <li
              key={agent.id}
              className="border-outline-variant/30 bg-surface-container flex items-center justify-between gap-3 rounded-lg border px-3 py-2.5"
            >
              <div className="flex items-center gap-2">
                <Icon
                  name={agent.icon}
                  className="text-primary text-[18px]"
                />
                <div>
                  <p className="text-on-surface text-sm font-medium">
                    {t(`agent.${agent.id}.name` as const)}
                  </p>
                  <p className="text-on-surface-variant text-xs">
                    {t(`agent.${agent.id}.role` as const)}
                  </p>
                </div>
              </div>

```



```

"use client";

import { useEffect, useRef } from "react";
import { Icon } from "@components/icon";
import { useLanguage } from "@lib/i18n/language-provider";
import type { PipelineLog } from "@lib/types";
import { cn } from "@lib/utils";

const chipTone: Record<PipelineLog["level"], string> = {
  info: "bg-surface-bright text-on-surface-variant border-outline-variant/40",
  decision: "bg-primary/10 text-primary border-primary/20",
  warn: "bg-amber-400/10 text-amber-300 border-amber-400/20",
  error: "bg-error/10 text-error border-error/20",
  success: "bg-tertiary/10 text-tertiary border-tertiary/20",
};

const textTone: Record<PipelineLog["level"], string> = {
  info: "text-on-surface",
  decision: "text-on-surface",
  warn: "text-amber-200",
  error: "text-error",
  success: "text-on-surface",
};

export function LogStream({
  logs,
  running = false,
}: {
  logs: PipelineLog[];
  running?: boolean;
}) {
  const endRef = useRef<HTMLDivElement>(null);
  const { t } = useLanguage();

  useEffect(() => {
    endRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [logs.length]);

  return (
    <div className="border-outline-variant/50 bg-surface-container-lowest relative flex h-[400px] flex-col rounded-lg border">
      <div className="border-outline-variant/30 bg-surface-container-low flex items-center justify-between rounded-t-lg border-b px-4 py-3">
        <div className="flex items-center gap-2">
          <span className="relative flex h-2 w-2">
            <span
              className={cn(
                "absolute inline-flex h-full w-full rounded-full opacity-75",
                running ? "animate-ping bg-tertiary" : "bg-outline"
              )}
            />
            <span
              className={cn(
                "relative inline-flex h-2 w-2 rounded-full",
                running ? "bg-tertiary" : "bg-outline"
              )}
            />
          </span>
          <h3 className="font-label-sm text-on-surface text-[12px] uppercase tracking-wide">

```

```

        {t("logStream.title")}
      </h3>
    </div>
    <div className="flex gap-2 text-outline">
      <Icon name="filter_list" className="text-[16px]" />
      <Icon name="clear_all" className="text-[16px]" />
    </div>
  </div>

  {!!logs.length ? (
    <div className="flex flex-1 flex-col items-center justify-center px-6 text-center">
      <p className="text-on-surface text-sm font-medium">
        {t("logStream.waitingTitle")}
      </p>
      <p className="text-on-surface-variant mt-1 max-w-sm text-xs leading-5">
        {t("logStream.waitingDesc")}
      </p>
    </div>
  ) : (
    <div className="terminal-scroll flex-1 overflow-y-auto p-2">
      {logs.map((log) => (
        <div
          key={log.id}
          className={cn(
            "hover:bg-surface-container/50 group flex flex-col gap-1 rounded p-2 transition-colors",
            log.level === "error" &&
            "border-error bg-error-container/10 ml-1 border-l-2"
          )}
        >
          <div className="flex flex-wrap items-center gap-2">
            <span className="font-code-md text-outline text-[10px]">
              {new Date(log.ts).toLocaleTimeString(undefined, {
                hour12: false,
              })}
            </span>
            <span
              className={cn(
                "font-label-sm rounded border px-1 text-[10px]",
                chipTone[log.level]
              )}
            >
              {log.agent}
            </span>
            {log.field ? (
              <span className="font-code-md text-outline text-[10px]">
                {t("logStream.field")}:{log.field}
              </span>
            ) : null}
            {typeof log.confidence === "number" ? (
              <span className="font-code-md text-outline text-[10px]">
                {t("logStream.conf")}: {log.confidence.toFixed(2)}
              </span>
            ) : null}
          </div>
          <span
            className={cn("font-code-md text-[13px]", textTone[log.level])}
          >
            {log.message}
          </span>
          {log.evidence ? (
            <div
              className={cn(

```

```

        "font-code-md pl-[90px] text-[12px]",
        log.level === "error" ? "text-error/80" : "text-on-surface-variant/70"
    )}
    >
    &gt; {log.evidence}
  </div>
) : null}
</div>
)}}
{running ? (
  <div className="flex items-center gap-3 p-2 opacity-60">
    <span className="font-code-md text-outline text-[10px]">&gt;_</span>
    <span className="bg-primary h-3 w-1.5 animate-pulse" />
  </div>
) : null}
<div ref={endRef} />
</div>
)}
</div>
);
}

```

## src/components/result-dashboard.tsx

```

"use client";

import { Icon } from "@components/icon";
import { useLanguage } from "@lib/i18n/language-provider";
import type { AgentDecision, PipelineResult } from "@lib/types";
import { cn } from "@lib/utils";

const decisionTone: Record<AgentDecision, string> = {
  aprobado: "bg-tertiary/15 text-tertiary border-tertiary/30",
  "requiere revisión": "bg-amber-400/15 text-amber-200 border-amber-400/30",
  bloqueado: "bg-error/15 text-error border-error/30",
};

const decisionKey: Record<AgentDecision, `decision.${AgentDecision}`> = {
  aprobado: "decision.aprobado",
  "requiere revisión": "decision.requiere revisión",
  bloqueado: "decision.bloqueado",
};

function downloadJson(result: PipelineResult) {
  const blob = new Blob([JSON.stringify(result, null, 2)], {
    type: "application/json",
  });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = `helix-${result.runId}.json`;
  a.click();
  URL.revokeObjectURL(url);
}

async function downloadPdf(result: PipelineResult) {
  const res = await fetch("/api/export/pdf", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(result),
  });
}

```



```

});
if (!res.ok) throw new Error("No se pudo generar el PDF.");
const blob = await res.blob();
const url = URL.createObjectURL(blob);
const a = document.createElement("a");
a.href = url;
a.download = `helix-${result.runId}.pdf`;
a.click();
URL.revokeObjectURL(url);
}

export function ExportButtons({
  result,
  onPdfError,
}): {
  result: PipelineResult;
  onPdfError?: (message: string) => void;
} {
  const { t } = useLanguage();

  return (
    <div className="flex gap-2">
      <button
        type="button"
        onClick={() => downloadJson(result)}
        className="font-label-sm text-on-surface-variant hover:text-on-surface bg-surface-container border-
outline-variant/30 flex items-center gap-2 rounded border px-2 py-1 text-[12px] transition-colors"
      >
        <Icon name="data_object" className="text-[14px]" />{" "}
        {t("resultDashboard.exportJson")}
      </button>
      <button
        type="button"
        onClick={() => {
          downloadPdf(result).catch((err) =>
            onPdfError?.(err instanceof Error ? err.message : String(err))
          );
        }}
        className="font-label-sm text-on-surface-variant hover:text-on-surface bg-surface-container border-
outline-variant/30 flex items-center gap-2 rounded border px-2 py-1 text-[12px] transition-colors"
      >
        <Icon name="picture_as_pdf" className="text-[14px]" />{" "}
        {t("resultDashboard.exportPdf")}
      </button>
    </div>
  );
}

export function ResultDashboard({ result }: { result: PipelineResult }) {
  const { t } = useLanguage();
  const seconds = (result.durationMs / 1000).toFixed(1);
  const totalSteps = result.agents.length;
  const doneSteps = result.agents.filter((a) => a.status === "done").length;

  return (
    <div className="space-y-6">
      <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
        <StatCard label={t("resultDashboard.timeTotal")} value={` ${seconds}s` } />
        <StatCard
          label={t("resultDashboard.activeAgents")}
          value={String(result.activeAgents)}
        />
      </div>
    </div>
  );
}

```

```

<StatCard
  label={t("resultDashboard.stepsCompleted")}
  value={` ${doneSteps} / ${totalSteps}`}
/>
<StatCard
  label={t("resultDashboard.avgConfidence")}
  value={` ${Math.round(result.overallConfidence * 100)}%`}
  highlight
/>
</div>

<div className="bg-outline-variant/20 h-px w-full" />

<div className="overflow-hidden rounded-lg border border-outline-variant/30">
  <table className="w-full text-left text-sm">
    <thead className="bg-surface-container-low text-on-surface-variant text-[11px] tracking-wide uppercase">
      <tr>
        <th className="px-3 py-2 font-medium">
          {t("resultDashboard.tableAgent")}
        </th>
        <th className="px-3 py-2 font-medium">
          {t("resultDashboard.tableDecision")}
        </th>
        <th className="px-3 py-2 font-medium">
          {t("resultDashboard.tableConf")}
        </th>
        <th className="px-3 py-2 font-medium">
          {t("resultDashboard.tableSummary")}
        </th>
      </tr>
    </thead>
    <tbody>
      {result.agents.map((run) => {
        const name = t(`agent.${run.agent}.name` as const);
        const decision = run.decision ?? "requiere revisión";
        return (
          <tr key={run.agent} className="border-outline-variant/20 border-t">
            <td className="text-on-surface px-3 py-2.5 font-medium">
              {name}
            </td>
            <td className="px-3 py-2.5">
              <span
                className={cn(
                  "rounded-full border px-2 py-0.5 text-[11px]",
                  decisionTone[decision]
                )}
              >
                {t(decisionKey[decision])}
              </span>
            </td>
            <td className="font-code-md px-3 py-2.5 text-xs">
              {run.confidence.toFixed(2)}
            </td>
            <td className="text-on-surface-variant px-3 py-2.5 text-xs">
              {run.summary}
            </td>
          </tr>
        );
      })}
    </tbody>
  </table>

```

```

</div>

<div className="flex flex-wrap gap-2 text-[11px] text-on-surface-variant">
  <Chip>
    {t("resultDashboard.chipMin")} {result.thresholds.minConfidence.toFixed(2)}
  </Chip>
  <Chip>
    {t("resultDashboard.chipHitL")} {result.thresholds.hitLThreshold.toFixed(2)}
  </Chip>
  <Chip>Claude {result.claudeEnabled ? "on" : "off"}</Chip>
  <Chip>Supabase {result.supabaseEnabled ? "on" : "off"}</Chip>
</div>
</div>
);
}

function StatCard({
  label,
  value,
  highlight,
}: {
  label: string;
  value: string;
  highlight?: boolean;
}) {
  return (
    <div
      className={cn(
        "relative flex flex-col overflow-hidden rounded border p-3",
        highlight
          ? "border-tertiary/30 bg-surface-container"
          : "border-outline-variant/20 bg-surface-container"
      )}
    >
      {highlight ? <div className="bg-tertiary/5 absolute inset-0" /> : null}
      <span
        className={cn(
          "font-label-sm relative z-10 text-[10px] uppercase tracking-wider",
          highlight ? "text-tertiary" : "text-outline"
        )}
      >
        {label}
      </span>
      <span
        className={cn(
          "font-code-md relative z-10 mt-1 text-[20px]",
          highlight ? "text-tertiary font-bold" : "text-on-surface"
        )}
      >
        {value}
      </span>
    </div>
  );
}

function Chip({ children }: { children: React.ReactNode }) {
  return (
    <span className="border-outline-variant/30 rounded-full border px-2 py-0.5">
      {children}
    </span>
  );
}

```

```

"use client";

import { useMemo, useState } from "react";
import { Icon } from "@components/icon";
import { ResultDashboard } from "@components/result-dashboard";
import { useLanguage } from "@lib/i18n/language-provider";
import type { PipelineResult, ScoredField } from "@lib/types";
import { cn } from "@lib/utils";

function confidenceTone(value: number): string {
  if (value ≥ 0.8) return "text-tertiary";
  if (value ≥ 0.7) return "text-primary";
  return "text-amber-300";
}

function FieldRow({
  field,
  approved,
  onToggle,
}: {
  field: ScoredField;
  approved: boolean;
  onToggle: (key: string, next: boolean) ⇒ void;
}) {
  const { t } = useLanguage();
  const [open, setOpen] = useState(field.needsHuman);

  return (
    <div
      className={cn(
        "group relative flex flex-col rounded border p-3",
        field.needsHuman
          ? "border-[#d97706]/40 bg-surface-container"
          : "border-outline-variant/30 bg-surface-container"
      )}
    >
      <button
        type="button"
        onClick={() ⇒ setOpen((v) ⇒ !v)}
        className="flex items-center justify-between"
      >
        <div className="flex items-center gap-2">
          <span className="font-code-md text-secondary text-[13px]">
            {field.key}
          </span>
          {field.needsHuman ? (
            <span className="rounded bg-[#d97706]/20 px-1 text-[9px] font-bold uppercase text-[#d97706]">
              Review
            </span>
          ) : (
            <span className={cn("font-code-md text-[11px]", confidenceTone(field.confidence))}>
              {Math.round(field.confidence * 100)}%
            </span>
          )}
        </div>
        <Icon
          name={open ? "expand_less" : "expand_more"}
          className="text-outline group-hover:text-primary text-[18px]"
        />
      </button>
    >
      <ResultDashboard
        result={field.result}
        approved={approved}
        onToggle={onToggle}
      />
    </div>
  );
}

```

```

</button>

{open ? (
  <div className="border-outline-variant/20 mt-2 flex flex-col gap-2 border-t pt-2">
    <span
      className={cn(
        "font-body-md inline-block w-fit rounded p-1 text-[14px]",
        field.needsHuman
          ? "bg-error/10 text-error"
          : "text-on-surface"
      )}
    >
      &quot;{field.value}&quot;
    </span>
    <p className="text-on-surface-variant text-xs leading-5">
      {t("resultPanel.evidence")}: {field.evidence}
    </p>
    {field.needsHuman ? (
      <label className="mt-1 flex items-center gap-2 text-xs">
        <input
          type="checkbox"
          checked={approved}
          onChange={(e) => onToggle(field.key, e.target.checked)}
          className="text-[#d97706] focus:ring-0 border-outline-variant bg-surface rounded"
        />
        <span className="text-on-surface-variant font-label-sm">
          {t("resultPanel.approve")}
        </span>
      </label>
    ) : null}
  </div>
) : null}
</div>
);
}

export function ResultPanel({ result }: { result: PipelineResult }) {
  const { t } = useLanguage();
  const humanFields = useMemo(
    () => result.fields.filter((f) => f.needsHuman),
    [result.fields]
  );
  const [approved, setApproved] = useState<Record<string, boolean>>({});
  const [signedOff, setSignedOff] = useState(false);

  const allApproved =
    humanFields.length === 0 || humanFields.every((f) => approved[f.key]);
  const missingCount = humanFields.filter((f) => !approved[f.key]).length;

  return (
    <div className="space-y-6">
      <ResultDashboard result={result} />
      {result.sourceUrl ? (
        <p className="text-on-surface-variant text-xs break-all">
          {t("resultPanel.source")}: {result.sourceUrl}
        </p>
      ) : null}

      {result.humanRequired ? (
        <div className="relative flex items-start gap-4 overflow-hidden rounded-lg border border-[#d97706]/50 bg-[#1a1500] p-4">
          <div className="absolute top-0 bottom-0 left-0 w-1 bg-[#d97706]" />

```

```

<Icon name="warning" className="text-[24px] text-[#d97706]" />
<div className="flex-1">
  <h4 className="text-[16px] font-bold text-[#fcd34d]">
    {t("resultPanel.hitlTitle")}
  </h4>
  <p className="text-on-surface-variant mb-3 mt-1 text-[14px]">
    {t("resultPanel.hitlDesc", { count: humanFields.length })}
  </p>
  <button
    type="button"
    disabled={!allApproved}
    onClick={() => setSignedOff(true)}
    className={cn(
      "flex items-center gap-2 rounded px-4 py-2 text-[13px] font-bold transition-colors",
      allApproved
        ? "bg-[#d97706] text-black hover:bg-[#f59e0b]"
        : "bg-[#d97706]/30 text-black/60 cursor-not-allowed"
    )}
  >
    <Icon name="draw" className="text-[16px]" />
    {signedOff
      ? t("resultPanel.signed")
      : allApproved
      ? t("resultPanel.sign")
      : t("resultPanel.missingConfirmations", { count: missingCount })}
  </button>
</div>
</div>
) : null}

<div className="grid grid-cols-1 gap-8 md:grid-cols-2">
  <div className="flex flex-col gap-3">
    <h3 className="border-outline-variant/30 text-primary border-b pb-2 text-[12px] uppercase tracking-
widest">
      {t("resultPanel.fieldsTitle")}
    </h3>
    <div className="flex flex-col gap-2">
      {result.fields.map((field) => (
        <FieldRow
          key={field.key}
          field={field}
          approved={!approved[field.key]}
          onToggle={({key, next}) =>
            setApproved((prev) => ({ ...prev, [key]: next }))
          }
        />
      ))}
    </div>
  </div>

  <div className="flex flex-col gap-3">
    <h3 className="border-outline-variant/30 text-primary border-b pb-2 text-[12px] uppercase tracking-
widest">
      {t("resultPanel.claimsTitle")}
    </h3>
    <div className="flex flex-col gap-2">
      {result.claims.map((claim, i) => (
        <div
          key={i}
          className={cn(
            "flex items-start gap-2 rounded p-2",
            claim.status === "supported"

```

```

        ? "bg-surface-container"
        : "border-error/30 bg-surface-container border"
    )}
  >
  <Icon
    name={claim.status === "supported" ? "verified" : "gpp_bad"}
    className={cn(
      "mt-0.5 text-[16px]",
      claim.status === "supported" ? "text-tertiary" : "text-error"
    )}
  />
  <div>
    <p className="text-on-surface text-[13px]">{claim.text}</p>
    <p
      className={cn(
        "font-code-md mt-0.5 text-[10px]",
        claim.status === "supported"
          ? "text-outline"
          : "text-error/80"
      )}
    >
      {claim.note} · {t("resultPanel.confidenceLabel")}{ " " }
      {Math.round(claim.confidence * 100)}%
    </p>
  </div>
</div>
)}}
{!result.claims.length ? (
  <p className="text-on-surface-variant text-xs">
    {t("resultPanel.noClaims")}
  </p>
) : null}
</div>

{result.recommendations.length ? (
  <div className="mt-4 flex flex-col gap-2">
    <h4 className="text-on-surface-variant font-label-sm text-[11px] uppercase">
      {t("resultPanel.recommendationsTitle")}
    </h4>
    <ol className="text-on-surface-variant list-inside list-decimal space-y-1 text-[13px]">
      {result.recommendations.map((rec) => (
        <li key={rec.title}>
          <span className="text-on-surface font-medium">
            {rec.title}.
          </span>{ " " }
          {rec.detail}
        </li>
      ))}
    </ol>
  </div>
) : null}

{result.reviews.length ? (
  <div className="mt-4 flex flex-col gap-2">
    <h4 className="text-on-surface-variant font-label-sm text-[11px] uppercase">
      {t("resultPanel.reviewsTitle")}
    </h4>
    <ul className="space-y-1.5">
      {result.reviews.map((note, i) => (
        <li key={i} className="text-on-surface-variant text-[13px]">
          <span
            className={cn(

```

```

        "mr-2 rounded px-1.5 py-0.5 text-[10px] font-bold uppercase",
        note.verdict === "approve" &&
        "bg-tertiary/15 text-tertiary",
        note.verdict === "revise" &&
        "bg-amber-400/15 text-amber-200",
        note.verdict === "block" && "bg-error/15 text-error"
    )}
  >
    {note.verdict}
  </span>
  <span className="text-on-surface font-medium">
    {note.targetAgent}:
  </span>{" "}
  {note.comment}
</li>
  )}
</ul>
</div>
) : null}
</div>
</div>
</div>
);
}

```

## Datos y permisos

### src/lib/permissions.ts

```

import type { AgentId, PermissionKey, PermissionMap } from "../types";

export const AGENT_CATALOG: {
  id: AgentId;
  short: string;
  icon: string;
  accent: string;
}[] = [
  { id: "orchestrator", short: "ORCH", icon: "memory", accent: "cyan" },
  { id: "extractor", short: "EXT", icon: "save_as", accent: "sky" },
  { id: "seo", short: "SEO", icon: "search_insights", accent: "violet" },
  { id: "factcheck", short: "FACT", icon: "fact_check", accent: "amber" },
  { id: "recommender", short: "REC", icon: "recommend", accent: "emerald" },
  { id: "reviewer", short: "REV", icon: "rule", accent: "rose" },
];

export const PERMISSION_KEYS: PermissionKey[] = [
  "read_input",
  "fetch_url",
  "write_structured",
  "analyze_seo",
  "verify_claims",
  "write_recommendations",
  "peer_review",
  "request_human",
];

export const DEFAULT_PERMISSIONS: PermissionMap = {
  orchestrator: ["read_input", "fetch_url", "request_human", "peer_review"],
}

```



```

    extractor: ["read_input", "write_structured"],
    seo: ["read_input", "analyze_seo"],
    factcheck: ["read_input", "verify_claims"],
    recommender: ["read_input", "write_recommendations"],
    reviewer: ["read_input", "peer_review", "request_human"],
  };

export function can(
  permissions: PermissionMap,
  agent: AgentId,
  key: PermissionKey
): boolean {
  return permissions[agent]?.includes(key) ?? false;
}

```

## src/lib/config.ts

```

import type { AgentEnabledMap, AgentId, PermissionMap } from "./types";
import { DEFAULT_PERMISSIONS } from "./permissions";

export const DEFAULT_MIN_CONFIDENCE = 0.7;
export const DEFAULT_HITL_THRESHOLD = 0.85;

export const DEFAULT_ENABLED_AGENTS: AgentEnabledMap = {
  orchestrator: true,
  extractor: true,
  seo: true,
  factcheck: true,
  recommender: true,
  reviewer: true,
};

export function isAgentOn(
  enabled: AgentEnabledMap | undefined,
  agent: AgentId
): boolean {
  if (!enabled) return true;
  return enabled[agent] !== false;
}

export function withDefaultRequest(partial: {
  text?: string;
  url?: string;
  permissions?: PermissionMap;
  enabledAgents?: AgentEnabledMap;
  minConfidence?: number;
  hitlThreshold?: number;
}) {
  return {
    text: partial.text,
    url: partial.url,
    permissions: partial.permissions ?? DEFAULT_PERMISSIONS,
    enabledAgents: { ...DEFAULT_ENABLED_AGENTS, ...partial.enabledAgents },
    minConfidence: clamp01(partial.minConfidence ?? DEFAULT_MIN_CONFIDENCE),
    hitlThreshold: clamp01(partial.hitlThreshold ?? DEFAULT_HITL_THRESHOLD),
  };
}

export function clamp01(n: number): number {
  if (Number.isNaN(n)) return 0;
}

```

```
    return Math.max(0, Math.min(1, n));  
  }  
}
```

## src/lib/types.ts

```
export type AgentId =  
  | "orchestrator"  
  | "extractor"  
  | "seo"  
  | "factcheck"  
  | "recommender"  
  | "reviewer";  
  
export type InputKind = "url" | "article" | "copy";  
  
export type PermissionKey =  
  | "read_input"  
  | "fetch_url"  
  | "write_structured"  
  | "analyze_seo"  
  | "verify_claims"  
  | "write_recommendations"  
  | "peer_review"  
  | "request_human";  
  
export type PermissionMap = Record<AgentId, PermissionKey[]>;  
  
export type AgentEnabledMap = Record<AgentId, boolean>;  
  
export type LogLevel = "info" | "decision" | "warn" | "error" | "success";  
  
export type PipelineLog = {  
  id: string;  
  ts: string;  
  agent: AgentId;  
  level: LogLevel;  
  message: string;  
  field?: string;  
  confidence?: number;  
  evidence?: string;  
};  
  
export type ScoredField = {  
  key: string;  
  label: string;  
  value: string;  
  confidence: number;  
  evidence: string;  
  needsHuman: boolean;  
};  
  
export type Claim = {  
  text: string;  
  status: "supported" | "unverified" | "conflicted";  
  confidence: number;  
  note: string;  
};  
  
export type Recommendation = {  
  title: string;
```

```

    detail: string;
    priority: "alta" | "media" | "baja";
    confidence: number;
  };

export type ReviewNote = {
  targetAgent: AgentId;
  verdict: "approve" | "revise" | "block";
  comment: string;
};

export type AgentDecision = "aprobado" | "requiere revisión" | "bloqueado";

export type AgentRun = {
  agent: AgentId;
  status: "idle" | "running" | "done" | "skipped" | "blocked";
  summary: string;
  confidence: number;
  decision?: AgentDecision;
};

export type PipelineThresholds = {
  minConfidence: number;
  hitlThreshold: number;
};

export type PipelineResult = {
  runId: string;
  startedAt: string;
  finishedAt: string;
  durationMs: number;
  inputKind: InputKind;
  sourceText: string;
  sourceUrl?: string;
  fields: ScoredField[];
  claims: Claim[];
  recommendations: Recommendation[];
  reviews: ReviewNote[];
  agents: AgentRun[];
  logs: PipelineLog[];
  overallConfidence: number;
  humanRequired: boolean;
  activeAgents: number;
  thresholds: PipelineThresholds;
  claudeEnabled: boolean;
  supabaseEnabled: boolean;
};

export type AnalyzeRequest = {
  text?: string;
  url?: string;
  permissions: PermissionMap;
  enabledAgents: AgentEnabledMap;
  minConfidence: number;
  hitlThreshold: number;
};

export type StreamEvent =
  | { type: "log"; log: PipelineLog }
  | { type: "agent"; run: AgentRun }
  | { type: "result"; result: PipelineResult }
  | { type: "error"; message: string };

```

src/app/workflows/page.tsx

```
"use client";

import Link from "next/link";
import { Icon } from "@components/icon";
import { PageFooter } from "@components/page-footer";
import { TopNav } from "@components/top-nav";
import { useLanguage } from "@lib/i18n/language-provider";
import type { TranslationKey } from "@lib/i18n/dictionaries";

type StepKey = 1 | 2 | 3 | 4 | 5 | 6 | 7;

const STEP_KEYS: { key: StepKey; icon: string; parallel?: boolean }[] = [
  { key: 1, icon: "route" },
  { key: 2, icon: "save_as" },
  { key: 3, icon: "call_split", parallel: true },
  { key: 4, icon: "rule" },
  { key: 5, icon: "recommend" },
  { key: 6, icon: "rule" },
  { key: 7, icon: "merge" },
];

const INPUT_KINDS: { key: "url" | "article" | "copy"; icon: string }[] = [
  { key: "url", icon: "link" },
  { key: "article", icon: "article" },
  { key: "copy", icon: "short_text" },
];

export default function WorkflowsPage() {
  const { t } = useLanguage();

  return (
    <div className="text-on-surface font-body-md flex min-h-full flex-col">
      <TopNav active="workflows" />
      <main className="mx-auto flex w-full max-w-[1440px] flex-1 flex-col gap-10 px-4 py-8 pt-24 sm:px-6">
        <header>
          <p className="text-primary glow-text-primary mb-2 text-[12px] uppercase tracking-widest">
            {t("workflowsPage.kicker")}
          </p>
          <h1 className="text-on-surface text-[32px] font-bold leading-10">
            {t("workflowsPage.title")}
          </h1>
          <p className="text-on-surface-variant mt-2 max-w-2xl text-sm leading-6">
            {t("workflowsPage.subtitle")}
          </p>
        </header>

        {/* Vertical step timeline */}
        <section className="flex flex-col gap-3">
          {STEP_KEYS.map((step, i) => (
            <div key={step.key} className="flex gap-4">
              <div className="flex flex-col items-center">
                <span
                  className={cnPrimaryBadge(step.parallel)}
                >
                <Icon name={step.icon} className="text-[18px]" />
              </span>
            </div>
          ))}
        </section>
      </main>
    </div>
  );
}
```

```

        {i < STEP_KEYS.length - 1 ? (
          <div className="bg-outline-variant/30 mt-1 w-px flex-1" />
        ) : null}
      </div>
      <div className="border-outline-variant/30 bg-surface-container mb-3 flex-1 rounded-lg border
p-4">
        <div className="mb-1 flex flex-wrap items-center justify-between gap-2">
          <h3 className="text-on-surface text-sm font-semibold">
            {t(`workflowsPage.step${step.key}.title` as TranslationKey)}
          </h3>
          <span className="font-code-md text-outline text-[10px] uppercase">
            {t(`workflowsPage.step${step.key}.meta` as TranslationKey)}
          </span>
        </div>
        <p className="text-on-surface-variant text-[13px] leading-5">
          {t(`workflowsPage.step${step.key}.detail` as TranslationKey)}
        </p>
      </div>
    </div>
  )}}
</section>

{/* Input kinds */}
<section className="flex flex-col gap-3">
  <h2 className="text-on-surface flex items-center gap-2 text-[20px] font-semibold">
    <Icon name="category" className="text-primary text-[20px]" />
    {t("workflowsPage.inputKindsTitle")}
  </h2>
  <div className="grid grid-cols-1 gap-4 md:grid-cols-3">
    {INPUT_KINDS.map((kind) => (
      <div
        key={kind.key}
        className="border-outline-variant/30 bg-surface-container flex flex-col gap-2 rounded-lg border
p-4">
        <div className="flex items-center gap-2">
          <Icon name={kind.icon} className="text-primary text-[18px]" />
          <span className="font-code-md text-secondary text-[13px]">
            {t(`workflowsPage.kind.${kind.key}.label` as TranslationKey)}
          </span>
        </div>
        <p className="text-on-surface-variant text-[13px] leading-5">
          {t(`workflowsPage.kind.${kind.key}.detail` as TranslationKey)}
        </p>
      </div>
    ))}
  </div>
</section>

{/* Thresholds */}
<section className="border-outline-variant/30 bg-surface-container-low flex flex-col gap-3 rounded-lg
border p-5">
  <h2 className="text-on-surface flex items-center gap-2 text-[16px] font-semibold">
    <Icon name="tune" className="text-primary text-[18px]" />
    {t("workflowsPage.thresholdsTitle")}
  </h2>
  <div className="grid grid-cols-1 gap-4 sm:grid-cols-2">
    <div>
      <p className="font-code-md text-primary text-[13px]">
        {t("workflowsPage.minConfTitle")}
      </p>
      <p className="text-on-surface-variant mt-1 text-[13px] leading-5">

```

```

        {t("workflowsPage.minConfDetail")}
      </p>
    </div>
    <div>
      <p className="font-code-md text-[13px] text-[#fcd34d]">
        {t("workflowsPage.hitlTitle")}
      </p>
      <p className="text-on-surface-variant mt-1 text-[13px] leading-5">
        {t("workflowsPage.hitlDetail")}
      </p>
    </div>
  </div>
</section>

<Link
  href="/"
  className="text-primary w-fit text-sm font-medium hover:underline"
>
  {t("common.backToDashboard")}
</Link>
</main>
<PageFooter />
</div>
);
}

function cnPrimaryBadge(parallel?: boolean) {
  return [
    "flex size-9 shrink-0 items-center justify-center rounded-full border",
    parallel
      ? "border-secondary bg-secondary/15 text-secondary"
      : "border-primary bg-primary/15 text-primary",
  ].join(" ");
}

```

## Página Agents

src/app/agents/page.tsx

```

"use client";

import Link from "next/link";
import { Icon } from "@components/icon";
import { PageFooter } from "@components/page-footer";
import { TopNav } from "@components/top-nav";
import { useLanguage } from "@lib/i18n/language-provider";
import { AGENT_CATALOG, DEFAULT_PERMISSIONS } from "@lib/permissions";

export default function AgentsPage() {
  const { t } = useLanguage();

  return (
    <div className="text-on-surface font-body-md flex min-h-full flex-col">
      <TopNav active="agents" />
      <main className="mx-auto flex w-full max-w-[1440px] flex-1 flex-col gap-8 px-4 py-8 pt-24 sm:px-6">
        <header>
          <p className="text-primary glow-text-primary mb-2 text-[12px] uppercase tracking-widest">
            {t("agentsPage.kicker")}
          </p>
        </header>
      </main>
    </div>
  );
}

```

```

    </p>
    <h1 className="text-on-surface text-[32px] font-bold leading-10">
      {t("agentsPage.title")}
    </h1>
    <p className="text-on-surface-variant mt-2 max-w-2xl text-sm leading-6">
      {t("agentsPage.subtitle")}
    </p>
  </header>

  <div className="grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3">
    {AGENT_CATALOG.map((agent) => {
      const perms = DEFAULT_PERMISSIONS[agent.id] ?? [];
      return (
        <article
          key={agent.id}
          className="border-outline-variant/30 bg-surface-container flex flex-col gap-3 rounded-lg border
p-4"
        >
          <div className="flex items-center gap-3">
            <span className="bg-primary/15 text-primary flex size-9 shrink-0 items-center justify-center
rounded-lg">
              <Icon name={agent.icon} className="text-[20px]" />
            </span>
            <div>
              <p className="text-on-surface text-sm font-semibold">
                {t(`agent.${agent.id}.name` as const)}
              </p>
              <p className="font-code-md text-outline text-[10px] uppercase">
                {agent.short}
              </p>
            </div>
          </div>
          <p className="text-on-surface-variant text-[13px] leading-5">
            {t(`agent.${agent.id}.detail` as const)}
          </p>
          <div className="border-outline-variant/20 mt-auto border-t pt-3">
            <p className="text-on-surface-variant font-label-sm mb-2 text-[10px] uppercase tracking-
wide">
              {t("agentsPage.defaultPerms")}
            </p>
            <ul className="flex flex-wrap gap-1.5">
              {perms.length ? (
                perms.map((p) => (
                  <li
                    key={p}
                    className="border-outline-variant/30 text-on-surface-variant rounded-full border px-2
py-0.5 text-[11px]"
                  >
                    {t(`permission.${p}` as const)}
                  </li>
                ))
              ) : (
                <li className="text-outline text-[11px]">
                  {t("agentsPage.noDefaultPerms")}
                </li>
              )}
            </ul>
          </div>
        </article>
      );
    })}
  </div>

```

```

        <Link
          href="/"
          className="text-primary w-fit text-sm font-medium hover:underline"
        >
          {t("common.backToDashboard")}
        </Link>
      </main>
      <PageFooter />
    </div>
  );
}

```

## Página Logs

src/app/logs/page.tsx

```

"use client";

import Link from "next/link";
import { LogsClient } from "@components/logs-client";
import { PageFooter } from "@components/page-footer";
import { TopNav } from "@components/top-nav";
import { useLanguage } from "@lib/i18n/language-provider";

export default function LogsPage() {
  const { t } = useLanguage();

  return (
    <div className="text-on-surface font-body-md flex min-h-full flex-col">
      <TopNav active="logs" />
      <main className="mx-auto flex w-full max-w-[1440px] flex-1 flex-col gap-8 px-4 py-8 pt-24 sm:px-6">
        <header>
          <p className="text-primary glow-text-primary mb-2 text-[12px] uppercase tracking-widest">
            {t("logsPage.kicker")}
          </p>
          <h1 className="text-on-surface text-[32px] font-bold leading-10">
            {t("logsPage.title")}
          </h1>
          <p className="text-on-surface-variant mt-2 max-w-2xl text-sm leading-6">
            {t("logsPage.subtitle")}
          </p>
        </header>

        <LogsClient />

        <Link
          href="/"
          className="text-primary w-fit text-sm font-medium hover:underline"
        >
          {t("common.backToDashboard")}
        </Link>
      </main>
      <PageFooter />
    </div>
  );
}

```



```

"use client";

import { useCallback, useEffect, useState } from "react";
import { LogStream } from "@components/log-stream";
import { useLanguage } from "@lib/i18n/language-provider";
import type { PipelineLog } from "@lib/types";
import { cn } from "@lib/utils";

type RunSummary = { runId: string; lastTs: string; count: number };

async function fetchRuns(): Promise<{
  configured: boolean;
  runs: RunSummary[];
}> {
  const res = await fetch("/api/logs");
  const data = await res.json();
  return { configured: Boolean(data.configured), runs: data.runs ?? [] };
}

async function fetchLogsForRun(runId: string): Promise<PipelineLog[]> {
  const res = await fetch(`/api/logs?runId=${encodeURIComponent(runId)}`);
  const data = await res.json();
  return data.logs ?? [];
}

export function LogsClient() {
  const { t } = useLanguage();
  const [configured, setConfigured] = useState<boolean | null>(null);
  const [runs, setRuns] = useState<RunSummary[]>([]);
  const [selected, setSelected] = useState<string | null>(null);
  const [logs, setLogs] = useState<PipelineLog[]>([]);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    let active = true;
    fetchRuns().then(({ configured: isConfigured, runs: nextRuns }) => {
      if (!active) return;
      setConfigured(isConfigured);
      setRuns(nextRuns);
      setSelected((prev) => prev ?? nextRuns[0]?.runId ?? null);
    });
    return () => {
      active = false;
    };
  }, []);

  useEffect(() => {
    if (!selected) return;
    let active = true;
    fetchLogsForRun(selected).then((nextLogs) => {
      if (active) setLogs(nextLogs);
    });
    return () => {
      active = false;
    };
  }, [selected]);

  const refresh = useCallback(() => {
    setLoading(true);
  }, []);

```

```

fetchRuns()
  .then(({ configured: isConfigured, runs: nextRuns }) => {
    setConfigured(isConfigured);
    setRuns(nextRuns);
    setSelected((prev) => prev ?? nextRuns[0]?.runId ?? null);
  })
  .finally(() => setLoading(false));
}, []);

if (configured === null) {
  return (
    <p className="text-on-surface-variant text-sm">
      {t("logsClient.loading")}
    </p>
  );
}

if (!configured) {
  return (
    <div className="border-outline-variant/30 rounded-lg border border-dashed px-6 py-16 text-center">
      <p className="text-on-surface text-base font-medium">
        {t("logsClient.disabledTitle")}
      </p>
      <p className="text-on-surface-variant mx-auto mt-2 max-w-md text-sm leading-6">
        {t("logsClient.disabledDesc")}
      </p>
    </div>
  );
}

if (!runs.length) {
  return (
    <div className="border-outline-variant/30 rounded-lg border border-dashed px-6 py-16 text-center">
      <p className="text-on-surface text-base font-medium">
        {t("logsClient.noRunsTitle")}
      </p>
      <p className="text-on-surface-variant mx-auto mt-2 max-w-md text-sm leading-6">
        {t("logsClient.noRunsDesc")}
      </p>
    </div>
  );
}

return (
  <div className="flex flex-col gap-4">
    <div className="flex flex-wrap items-center justify-between gap-2">
      <div className="flex flex-wrap gap-2">
        {runs.map((run) => (
          <button
            key={run.runId}
            type="button"
            onClick={() => setSelected(run.runId)}
            className={cn(
              "font-code-md rounded-full border px-3 py-1 text-[11px] transition-colors",
              selected === run.runId
                ? "border-primary bg-primary/15 text-primary"
                : "border-outline-variant/30 text-on-surface-variant hover:text-on-surface"
            )}
          >
            {run.runId} · {run.count}
          </button>
        ))}
      </div>
    </div>
  </div>
);

```

```

        </div>
        <button
            type="button"
            onClick={refresh}
            disabled={loading}
            className="font-label-sm text-on-surface-variant hover:text-primary text-[12px] underline
disabled:opacity-50"
        >
            {t("logsClient.refresh")}
        </button>
    </div>
    <LogStream logs={logs} running={false} />
</div>
);
}

```

## src/app/api/logs/route.ts

```

import {
    isSupabaseConfigured,
    listLogsForRun,
    listRecentRuns,
} from "@lib/supabase-logs";

export const runtime = "nodejs";

export async function GET(req: Request) {
    const configured = isSupabaseConfigured();
    if (!configured) {
        return Response.json({ configured, runs: [], logs: [] });
    }

    const url = new URL(req.url);
    const runId = url.searchParams.get("runId");

    if (runId) {
        const logs = await listLogsForRun(runId);
        return Response.json({ configured, logs });
    }

    const runs = await listRecentRuns();
    return Response.json({ configured, runs });
}

```